

Transformer Circuit

트랜스포머를 해부해보자.

방학동안 무얼 하나요?

In a Nutshell

The list contains 27 reading items, with papers, blog posts, courses, one dissertation and two book chapters, all originally dating from 1993 to 2020.

The contents can be roughly broken down as follows:

Methodology	Items	Share*	Topics
Convolutional Neural Networks (CNNs)	5	25%	image recognition, semantic segmentation
Recurrent Neural Networks (RNNs)	10	19%	language modeling, speech-to-text, machine translation, combinatorial optimization, visual question answering, content-based attention
Transformers	3	6%	multi-head and dot-product attention, language model scaling
Information Theory	5	42%	Kolmogorov complexity, compression, Minimum Description Length
Miscellaneous	4	8%	variational inference, representation learning, graph neural networks, distributed training

til, Ilya sutskever gave john carmack this reading list of approx 30 research papers and said, ‘If you really learn all of these, you’ll know 90% of what matters today(2020).’

<https://tensorlabbet.com/2024/09/24/ai-reading-list/>

방학동안 무얼 하나요?

Q: 사칙연산을 어텐션 연산으로 어떻게 할까?

Q: 공간의 관점에서 E(mbedding), F(eature) row col space 사이 연산을 어떻게 이해할 수 있지?

Q: 1M와 같은 context를 어떻게 달성하지?

Q: 한 번에 여러 토큰을 생성하면 안되나?(Deepseek)

Q: 1bit transformer이 가능하다면, $f(x)$ bit 즉 자의적으로 사이즈를 조절하는 학습은?

방학동안 무얼 하나요?

- 사칙 연산을 틀리는 이유는 뭐지?(tokenization, tool?)

- context를 요약하면서 인간의 메모리와 비슷하게?

- Diffusion의 Noise 제거 관점에서 차용할 부분은?

Q: 사칙연산을 어텐션 연산으로 어떻게 할까?

- over-computing, 필요할 때만 자신의 크기를 키울 수 있을까?

Q: 공간의 관점에서 E(mbedding), F(eature) row col space 사이 연산을 어떻게 이해할 수 있지?

- 임베딩, 디임베딩의 이야기(GPT2)

Q: 1M와 같은 context를 어떻게 달성하지?

- positional embedding이 helix인 이유

Q: 한 번에 여러 토큰을 생성하면 안되나?(Deepseek)

- natural gradient??

Q: 1bit transformer이 가능하다면, $f(x)$ bit 즉 자의적으로 사이즈를 조절하는 학습은?

방학동안 무얼 하나요?

Language Models Use Trigonometry to Do Addition

Subhash Kantamneni¹ Max Tegmark¹

Abstract

Mathematical reasoning is an increasingly important indicator of large language model (LLM) capabilities, yet we lack understanding of how LLMs process even simple mathematical tasks. To address this, we reverse engineer how three mid-sized LLMs compute addition. We first discover that numbers are represented in these LLMs as a generalized helix, which is strongly causally implicated for the tasks of addition and subtraction, and is also causally relevant for integer division, multiplication, and modular arithmetic. We then propose that LLMs compute addition by manipulating this generalized helix using the “Clock” algorithm: to solve $a + b$, the helices for a and b are manipulated to produce the $a + b$ answer helix which is then read out to model logits. We model influential MLP outputs, attention head outputs, and even individual neuron preactivations with these helices and verify our understanding with causal interventions. By demonstrating that LLMs represent numbers on a helix and manipulate this helix to perform addition, we present the first representation-level explanation of an LLM’s

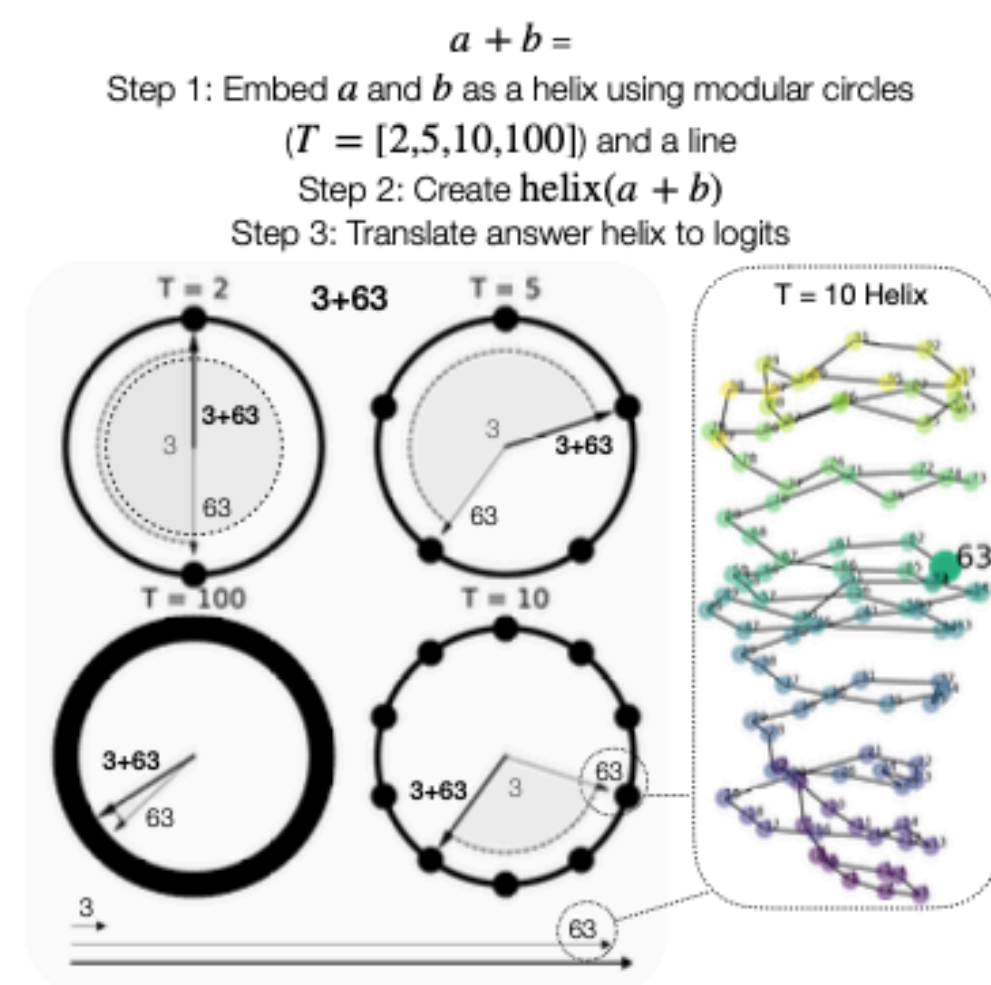


Figure 1. **Illustrating the Clock algorithm.** We find that LLMs represent numbers on a helix. When computing the addition problem $a + b$, LLMs rotate the a and b helices, as if on a clock, to create the $a + b$ helix and read out the final answer.

TREE ATTENTION: TOPOLOGY-AWARE DECODING FOR LONG-CONTEXT ATTENTION ON GPU CLUSTERS

Vasudev Shyam^{1*}, Jonathan Pilault^{1*}, Emily Shepperd², Quentin Anthony¹, Beren Millidge¹

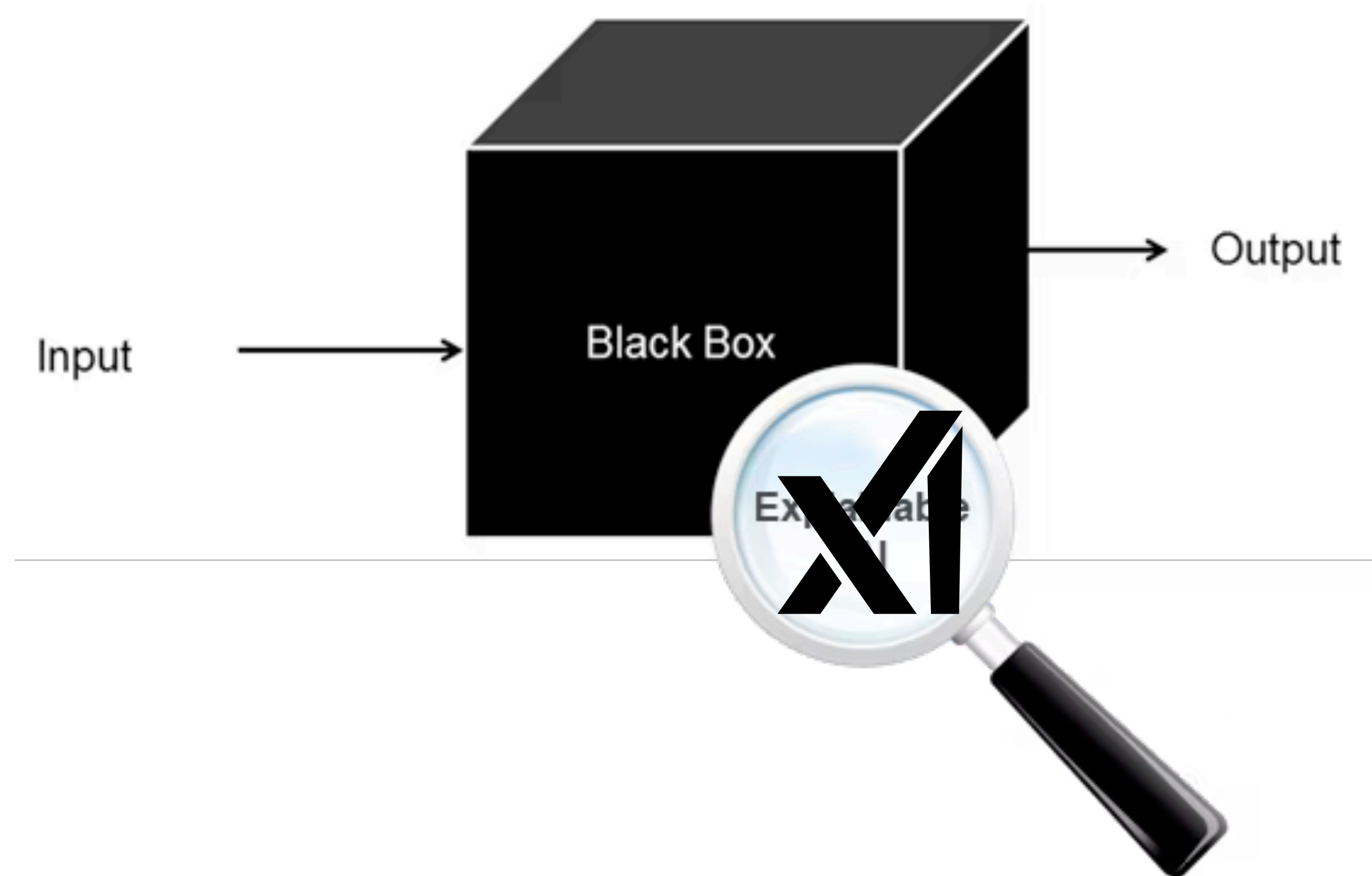
¹Zyphra, ²EleutherAI

jonathan.pilault@gmail.com, vasu@zyphra.com

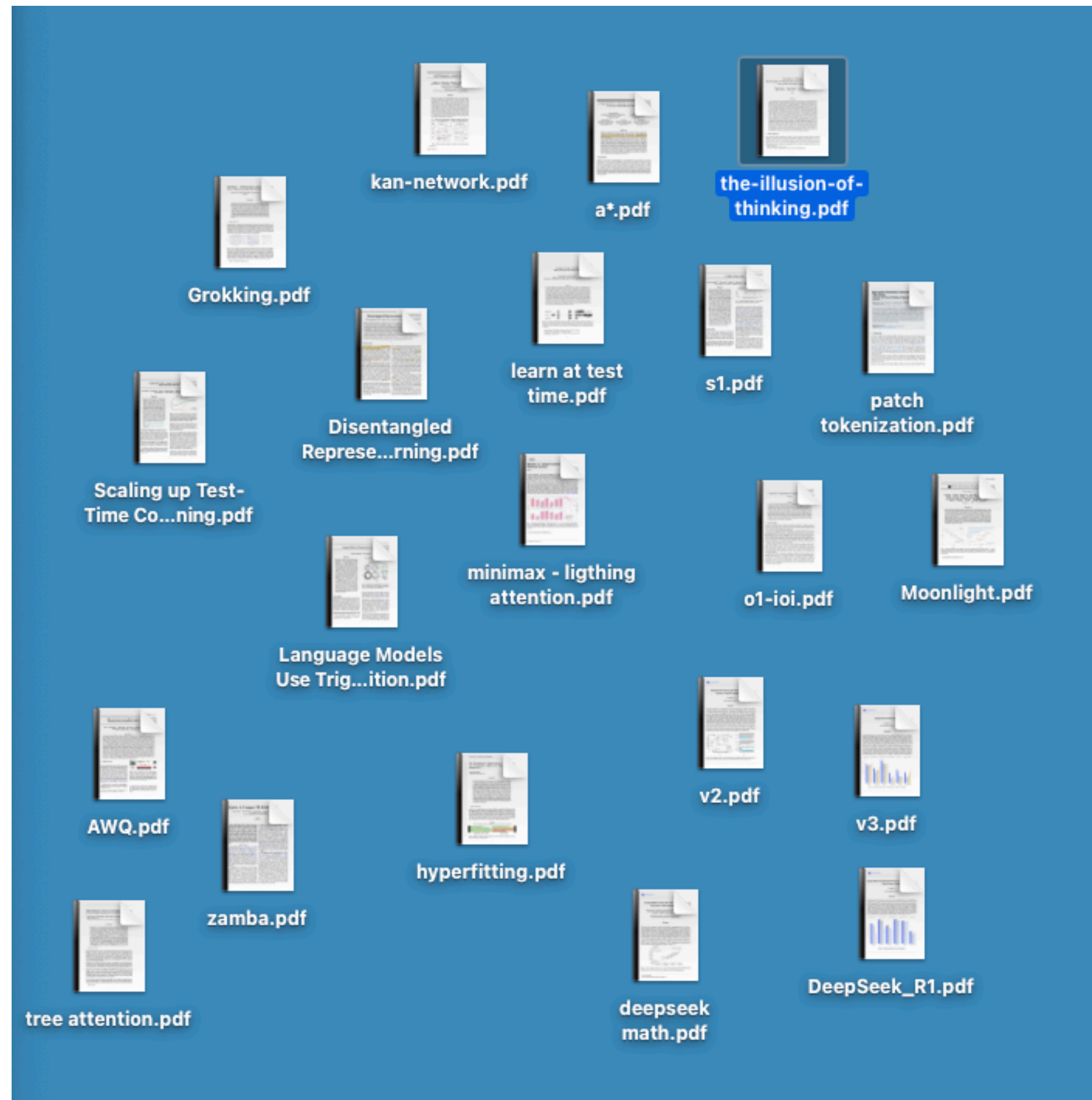
ABSTRACT

Self-attention is the core mathematical operation of modern transformer architectures and is also a significant computational bottleneck due to its quadratic complexity in the sequence length. In this work, we derive the scalar energy function whose gradient computes the self-attention block, thus elucidating the theoretical underpinnings of self-attention. Our formulation reveals that the reduction across the sequence axis can be efficiently computed in parallel through a tree reduction. Our algorithm, called *Tree Attention*, for parallelizing exact attention computation across multiple GPUs enables cross-device decoding to be performed *asymptotically* faster (up to 8× faster in our experiments) than state-of-the-art approaches such as *Ring Attention*, while also requiring significantly less communication volume and incurring 2× less peak memory. We demonstrate that *Tree Attention* speeds up decoding up to 4x on Llama 3.1-8B and can be applied to a variety of hardware and networking setups such as H100 DGX nodes, AMD MI300x nodes, and PCIe connected NVIDIA RTX 4090s. Our code is publicly available here: https://github.com/Zyphra/tree_attention

방학동안 무얼 하나요?



방학동안 무얼 하나요?



저의 경우는 바탕화면에
쌓인 수많은 Reading
List를 해결하고자 합니
다. 끔찍한 FOMO 오브제..

방학동안 무얼 하나요?

The Lost Reading Items

11 Nov 2024, Taro Langner

In this post: An attempt to reconstruct Ilya Sutskever's 2020 AI reading list
(8 min read)

메타러닝, RL, learning theory, ...

2학기부터.. 해봐요!1

What is convolution

1	1	1 _{x1}	0 _{x0}	0 _{x1}
0	1	1 _{x0}	1 _{x1}	0 _{x0}
0	0	1 _{x1}	1 _{x0}	1 _{x1}
0	0	1	1	0
0	1	1	0	0

Image

4	3	4

Convolved
Feature

- 흔하게 볼 수 있는 CNN 설명 그림

자연스레드는 질문들...

- 합성곱을 한다고? 이걸로 핫도그 여부를 판단할 수 있다고? 그게 그래서 왜 합성곱이여만 하지?
- 인간의 시신경 구조와 닮은 부분이 있다고? 어떤 부분이?

+인공지능이 어떻게 잘 배우게 함?

Inductive Bias

- 보기
- 읽기, 예측하기

+인공지능이 어떻게 잘 배우게 함?

Inductive Bias

- 보기

- 읽기, 예측하기

Spatial한 특징을 아까본 연산으로 살리기 어렵다

=> 2차원이라는 것 + 하나하나 픽셀보다 주변과의 관계 중요

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	1	12	0	11	39	137	37	0	152	147	84	0	0	0	0
0	0	1	0	0	0	41	160	250	255	235	162	255	238	206	11	13	0	0
0	0	0	16	9	9	150	251	45	21	184	159	154	255	233	40	0	0	0
10	0	0	0	0	0	145	146	3	10	0	11	124	253	255	107	0	0	0
0	0	3	0	4	15	236	216	0	0	38	109	247	240	169	0	11	0	0
1	0	2	0	0	0	253	253	23	62	224	241	255	164	0	5	0	0	0
6	0	0	4	0	3	252	250	228	255	255	234	112	28	0	2	17	0	0
0	2	1	4	0	21	255	253	251	255	172	31	8	0	1	0	0	0	0
0	0	4	0	163	225	251	255	229	120	0	0	0	0	0	11	0	0	0
0	0	21	162	255	255	254	255	126	6	0	10	14	6	0	0	9	0	0
3	79	242	255	141	66	255	245	189	7	8	0	0	5	0	0	0	0	0
26	221	237	98	0	67	251	255	144	0	8	0	0	7	0	0	11	0	0
125	255	141	0	87	244	255	208	3	0	0	13	0	1	0	1	0	0	0
145	248	228	116	235	255	141	34	0	11	0	1	0	0	0	1	3	0	0
85	237	253	246	255	210	21	1	0	1	0	0	6	2	4	0	0	0	0
6	23	112	157	114	32	0	0	0	0	2	0	8	0	7	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

+인공지능이 어떻게 잘 배우게 함?

Inductive Bias

- 보기

- 읽기, 예측하기

더 알아보기 -> CS231n Winter

α	β
γ	δ

applied to

A	B	C
D	E	F
G	H	J

yields

P	

α	β
γ	δ

A	B	C
D	E	F
G	H	J

	Q

α	β
γ	δ

A	B	C
D	E	F
G	H	J

R	

α	β
γ	δ

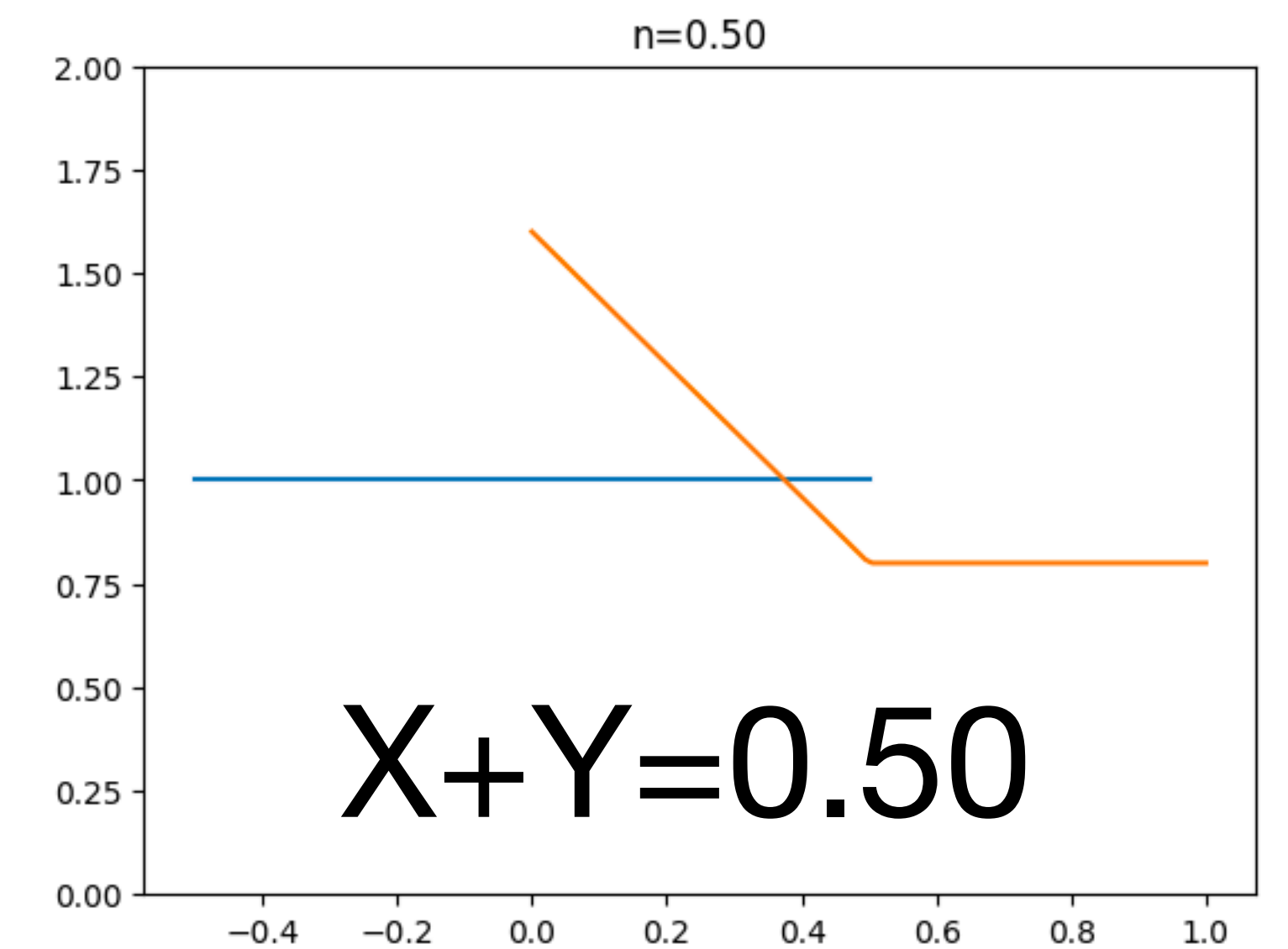
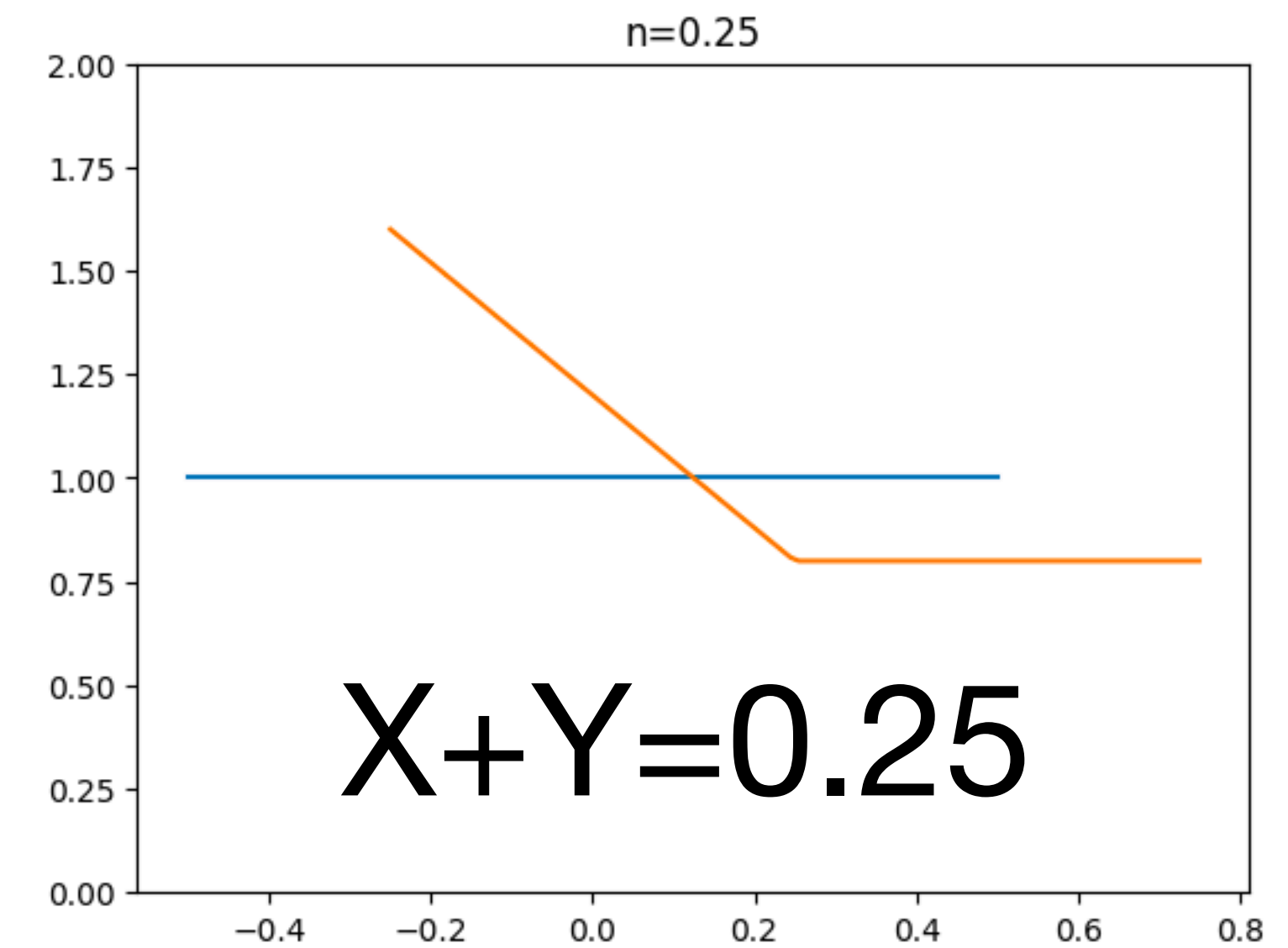
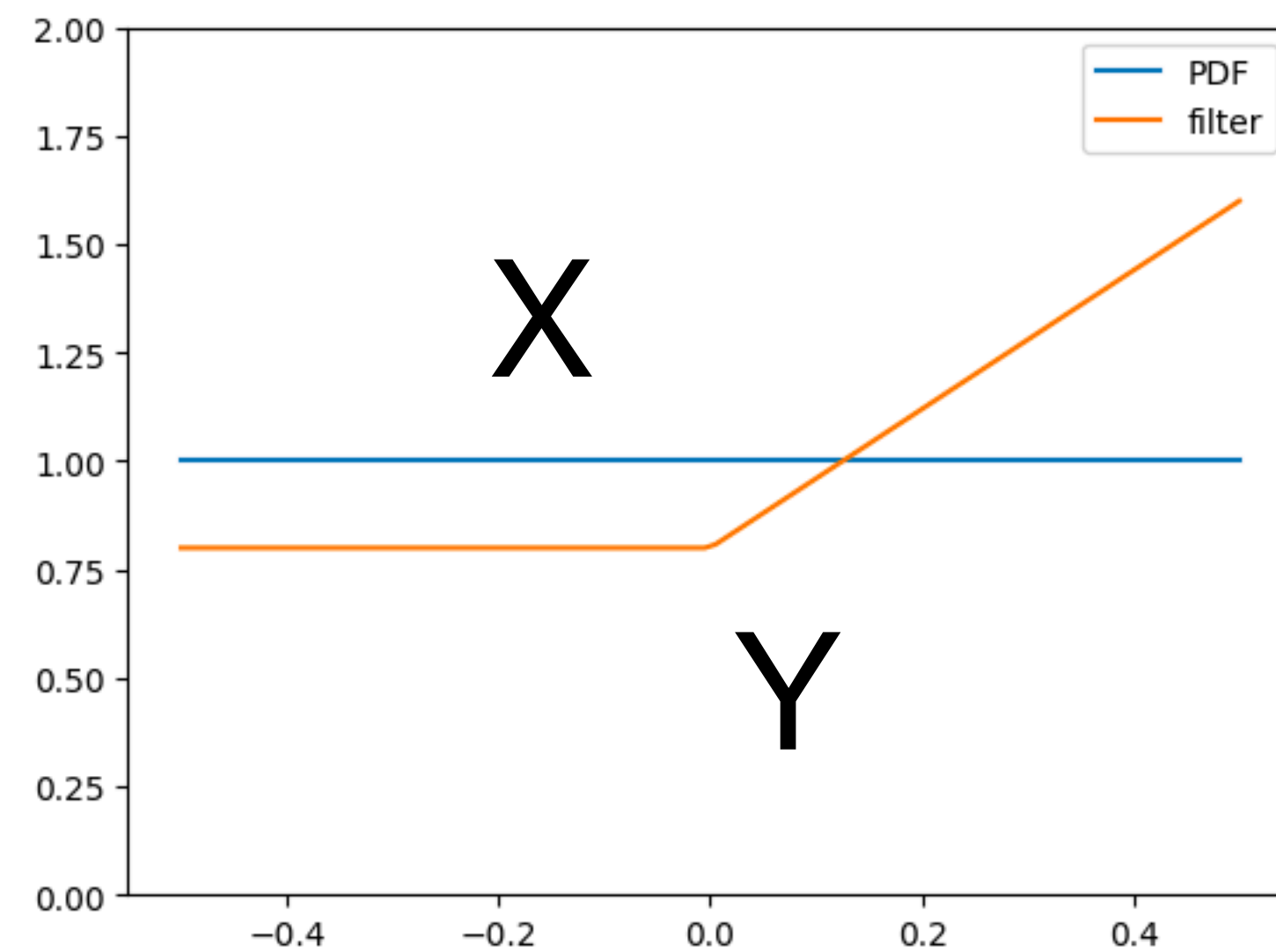
A	B	C
D	E	F
G	H	J

	S

100

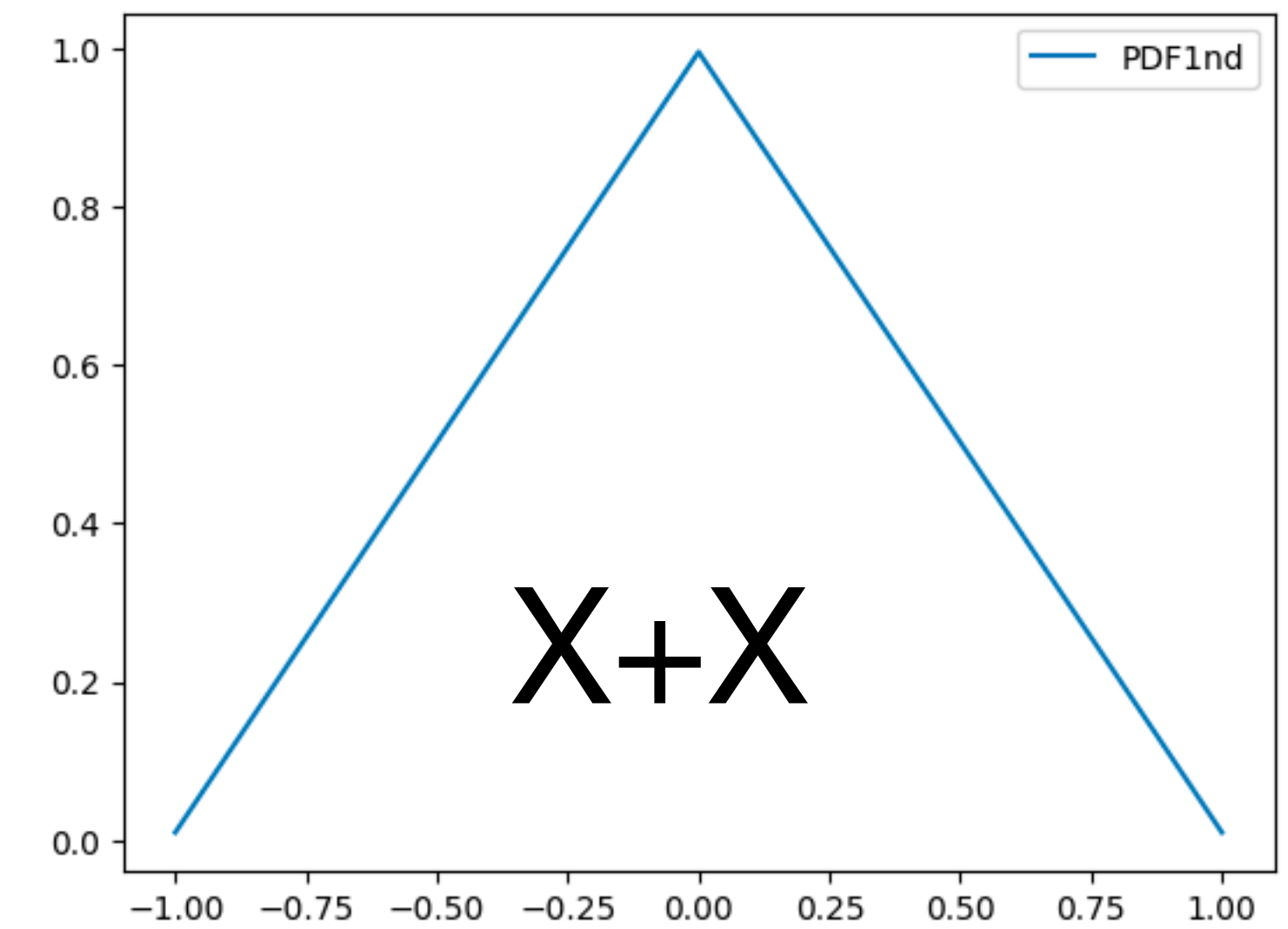
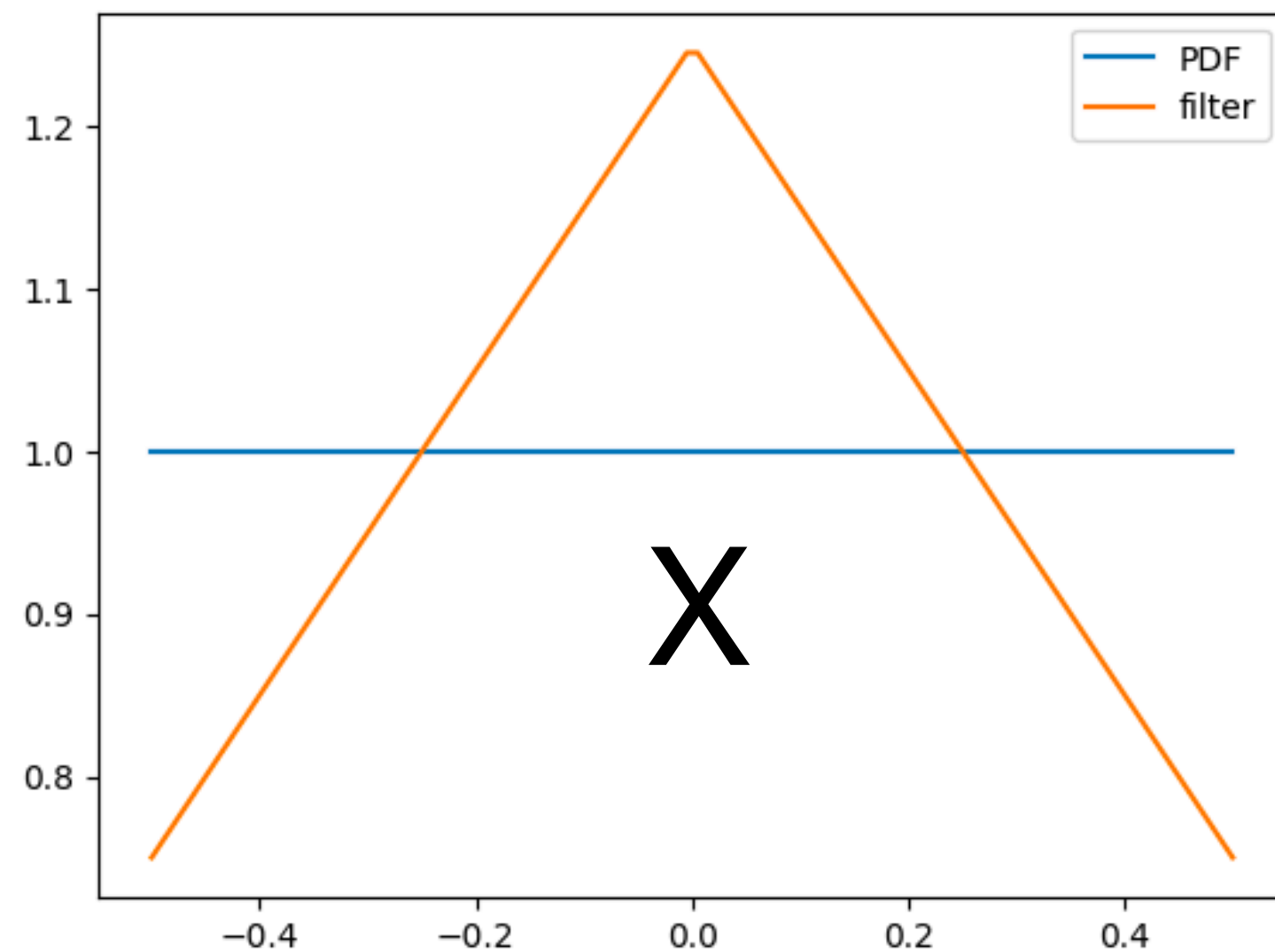
What is convolution

$$X+Y \quad \text{sum } f(t)g(n-t)$$



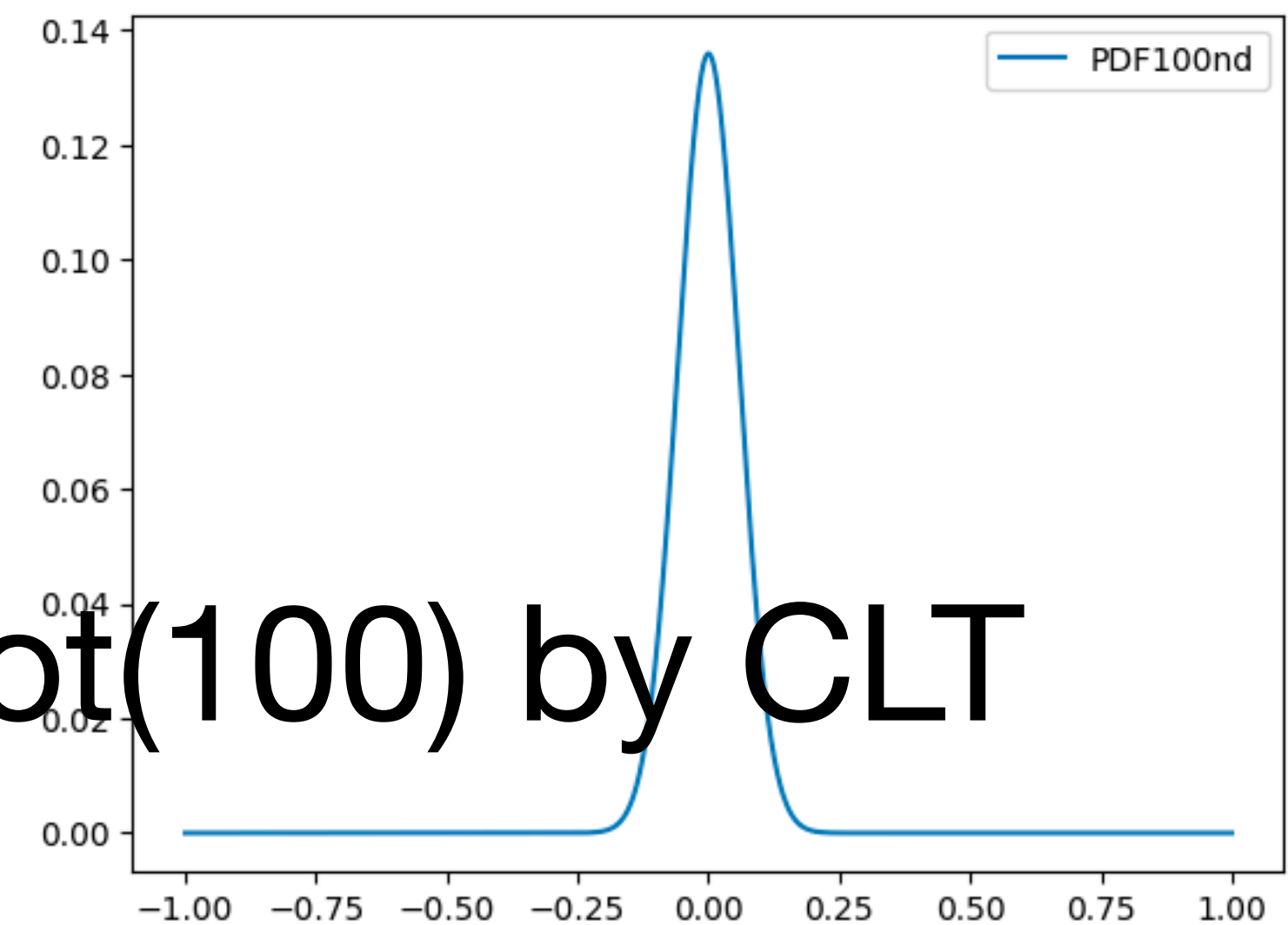
What is convolution

$$X+Y \quad \text{sum } f(t)g(n-t)$$



100X

$\sim N(0, \text{sigma}/\text{root}(100))$ by CLT

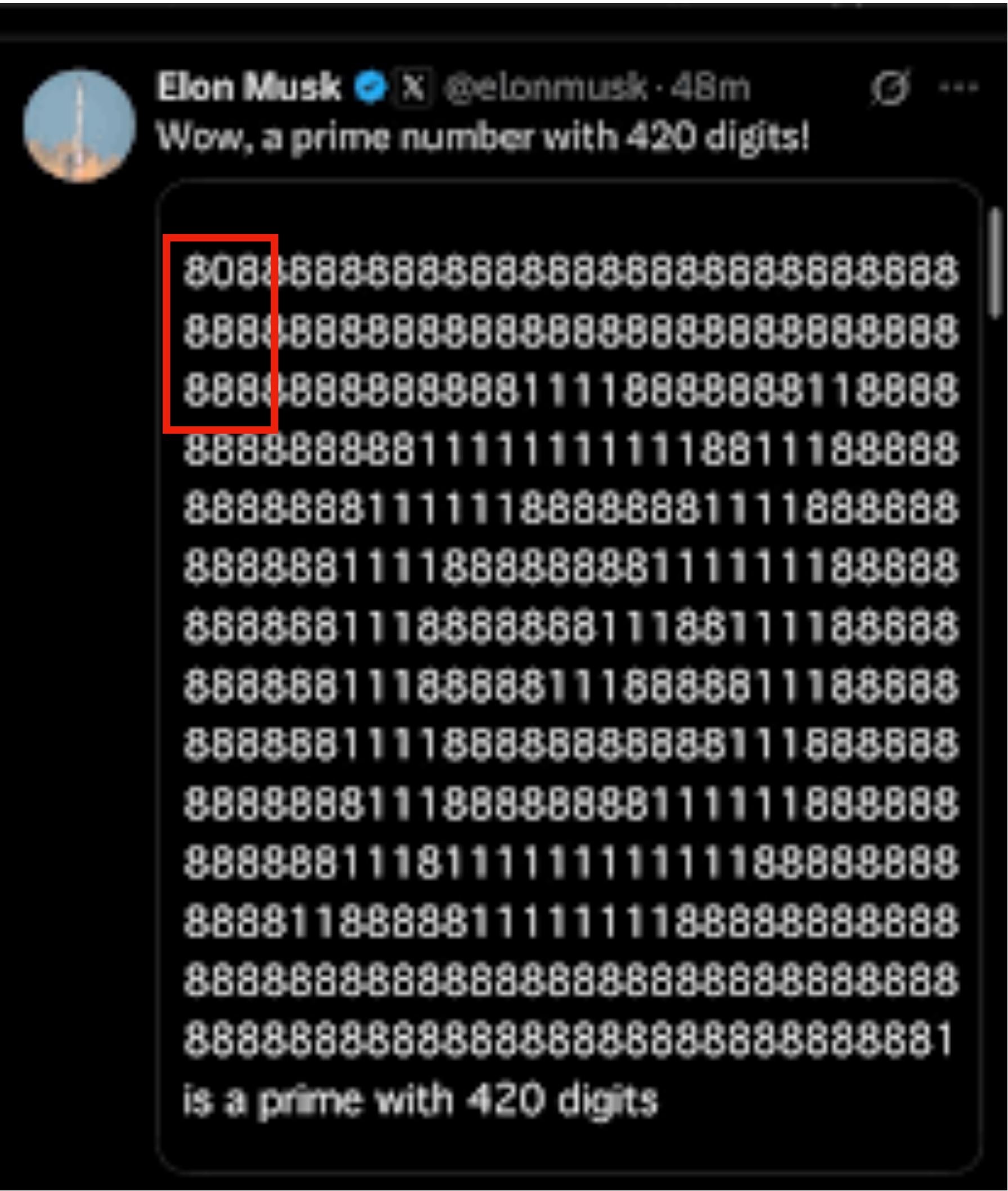
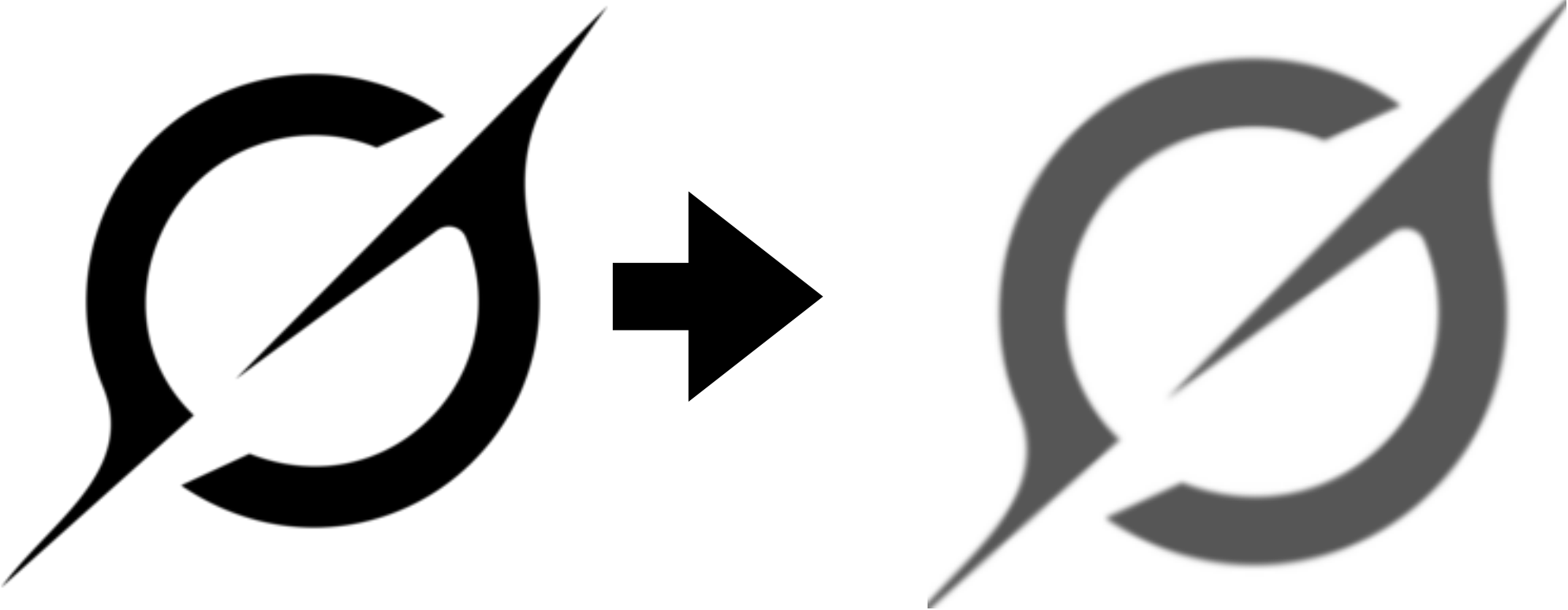


What is convolution

1/9 1/9 1/9

1/9 1/9 1/9

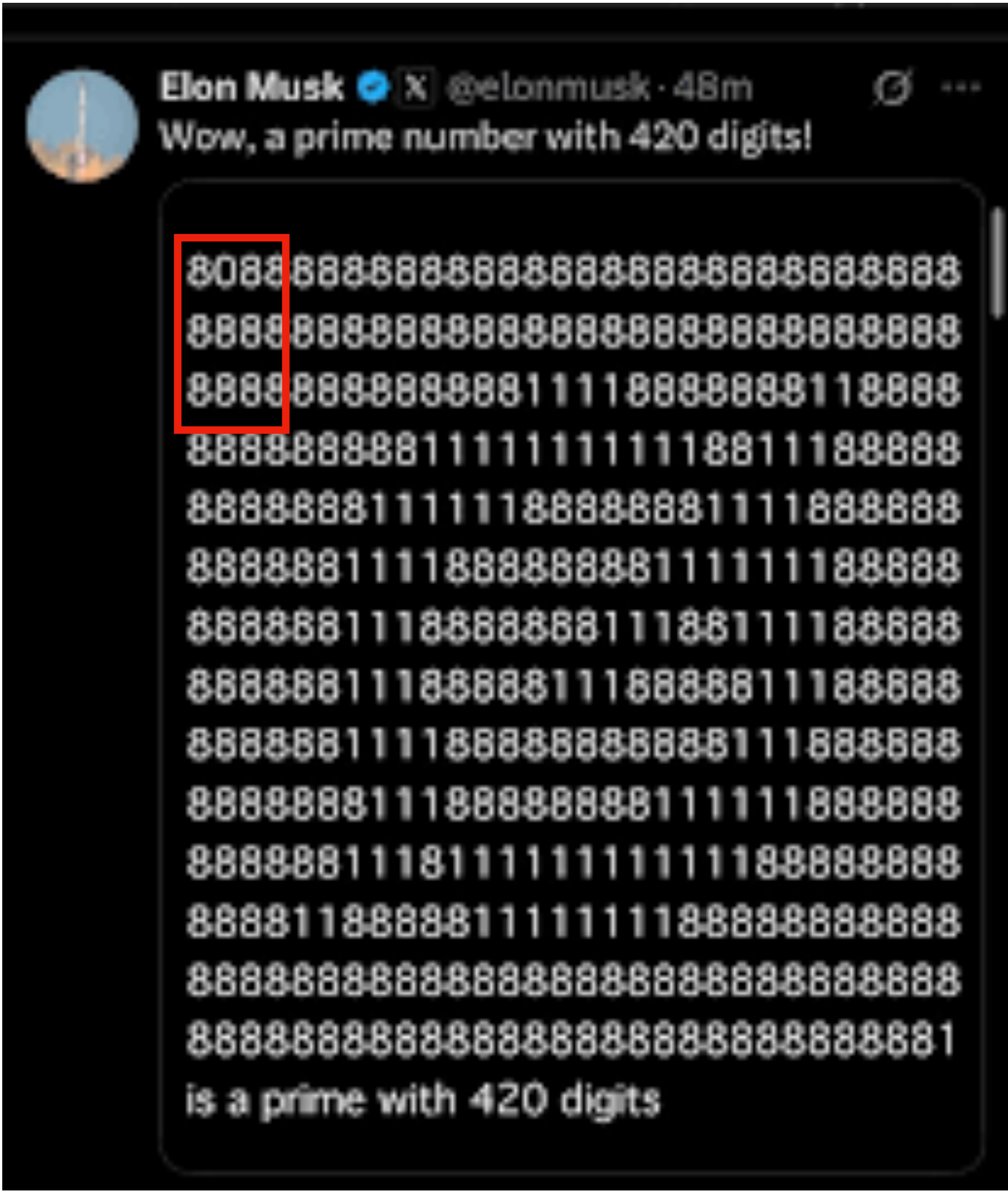
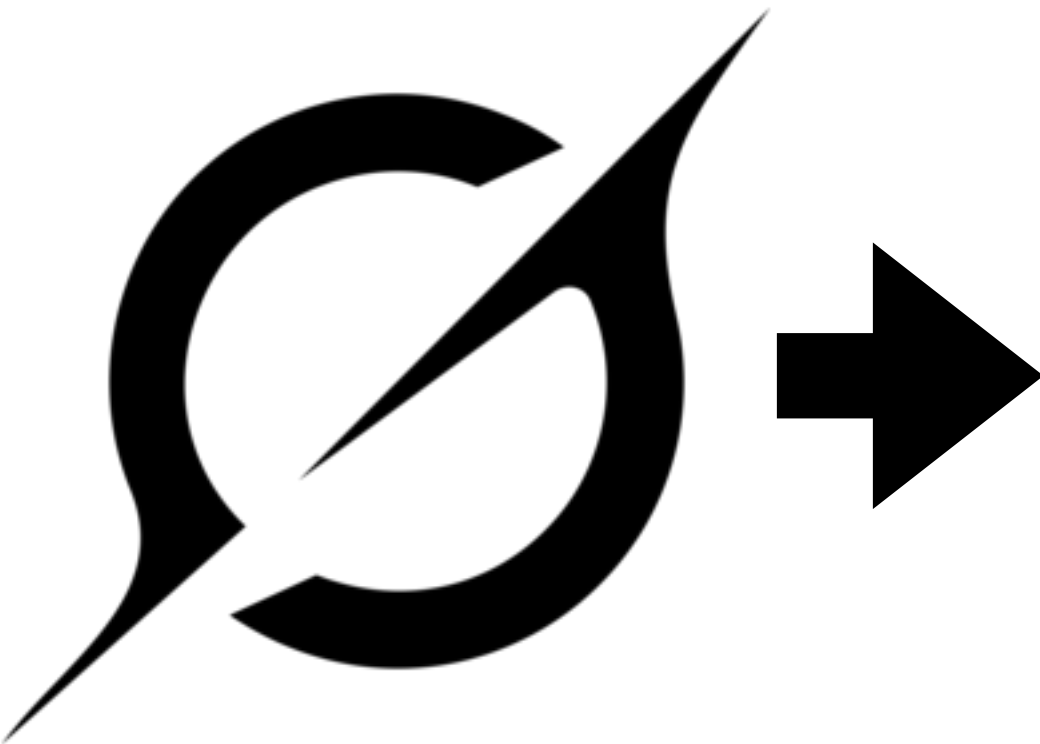
1/9 1/9 1/9



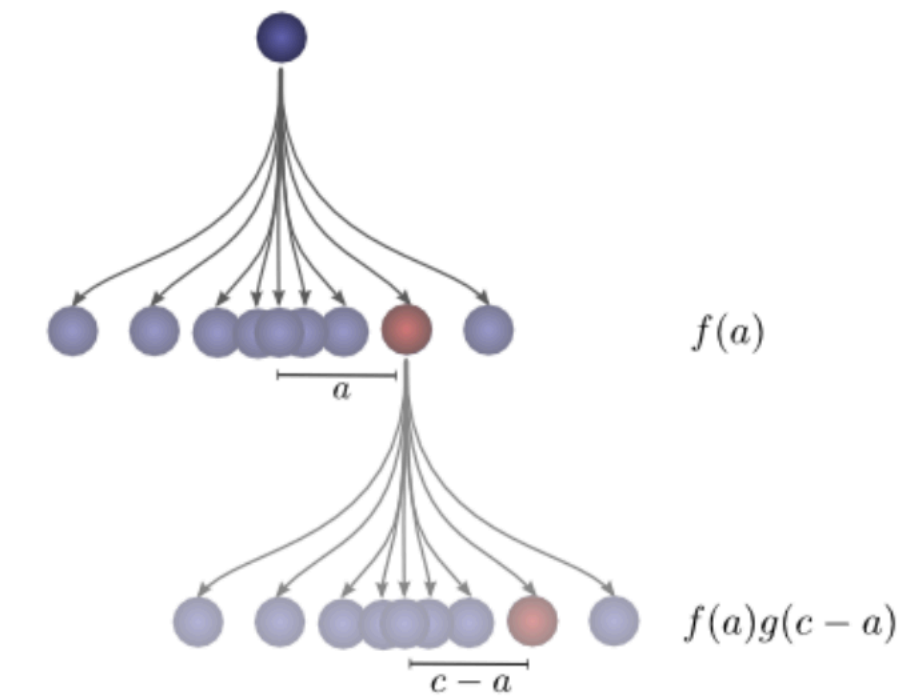
What is convolution

$\frac{1}{6}$ 0 $-\frac{1}{6}$
 $\frac{1}{6}$ 0 $-\frac{1}{6}$
 $\frac{1}{6}$ 0 $-\frac{1}{6}$

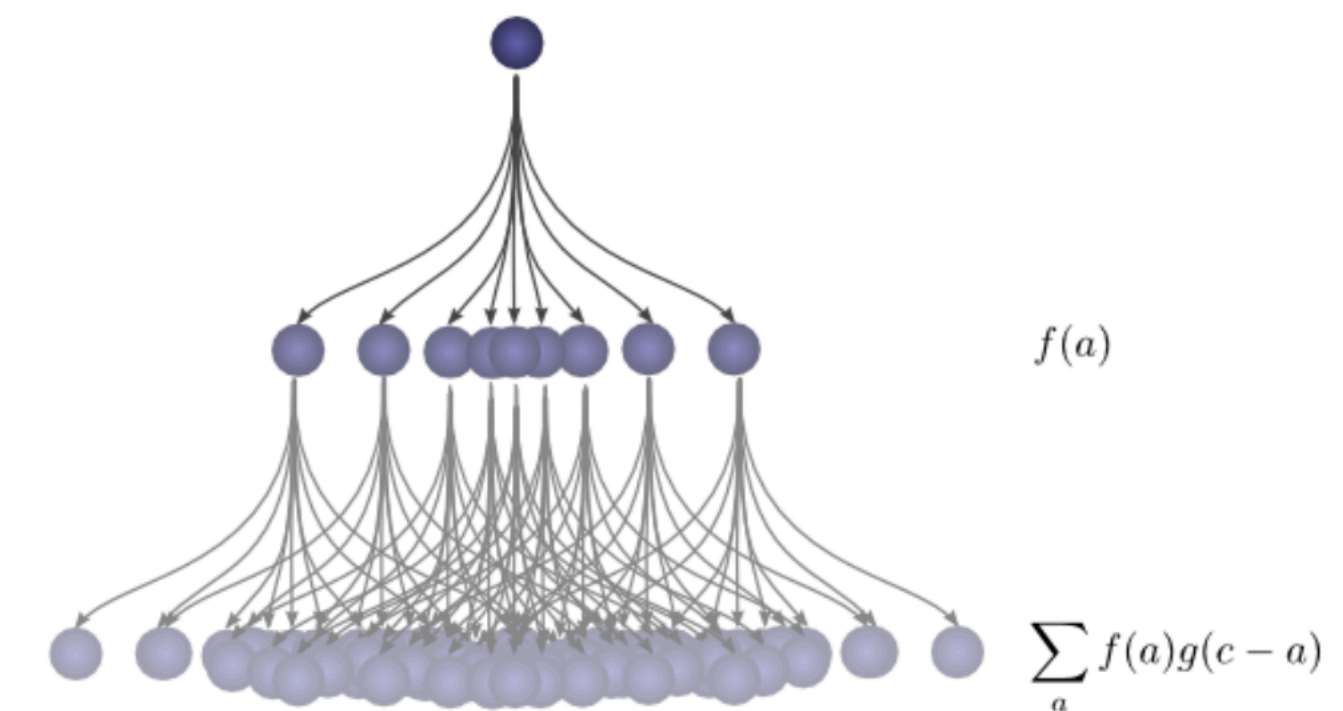
차이를 계산하는 필터!



What is convolution



To get the convolution, we consider all intermediate positions.



<https://colah.github.io/posts/2014-07-Understanding-Convolutions/>

So why convolution?

Best fit for image representation learning!

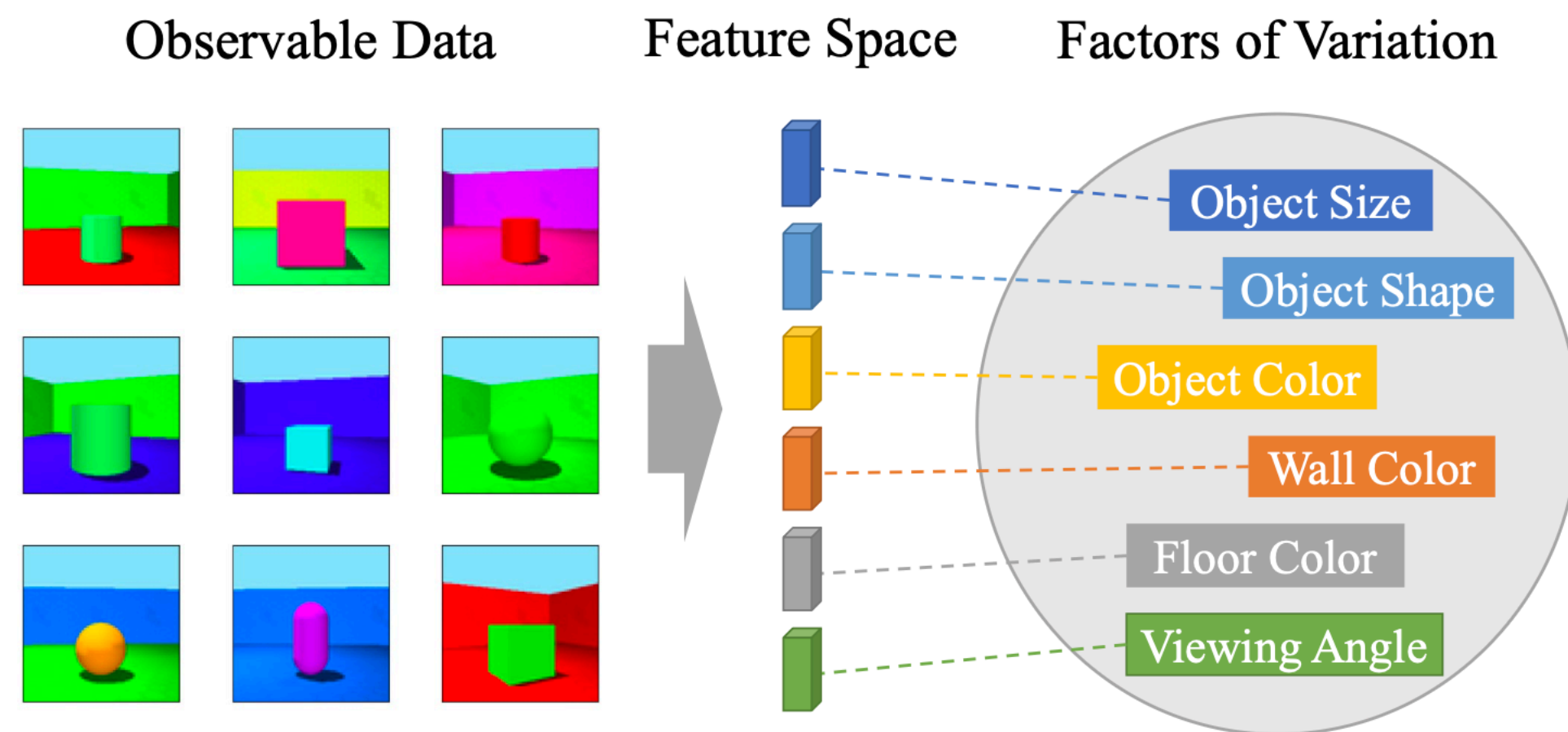
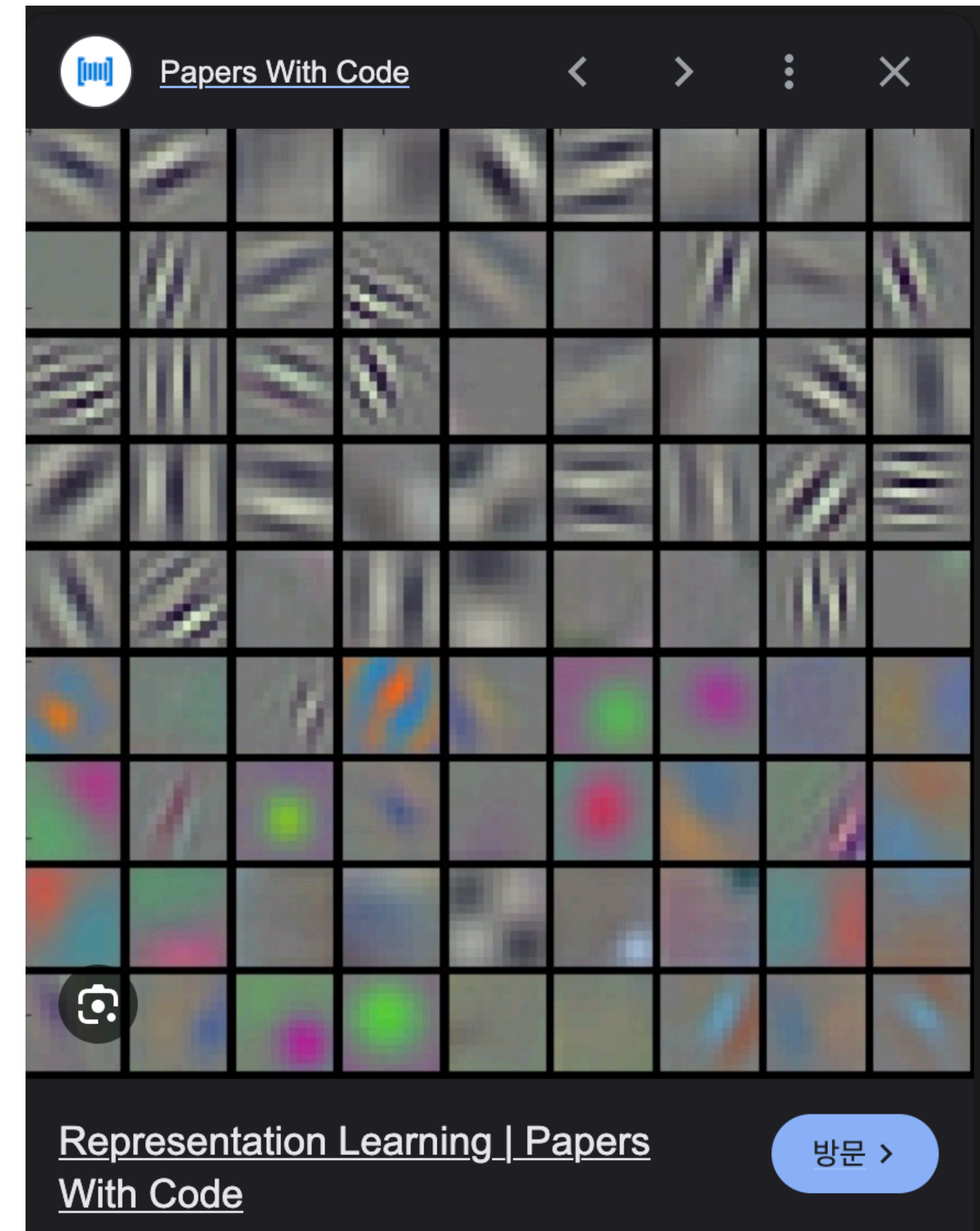
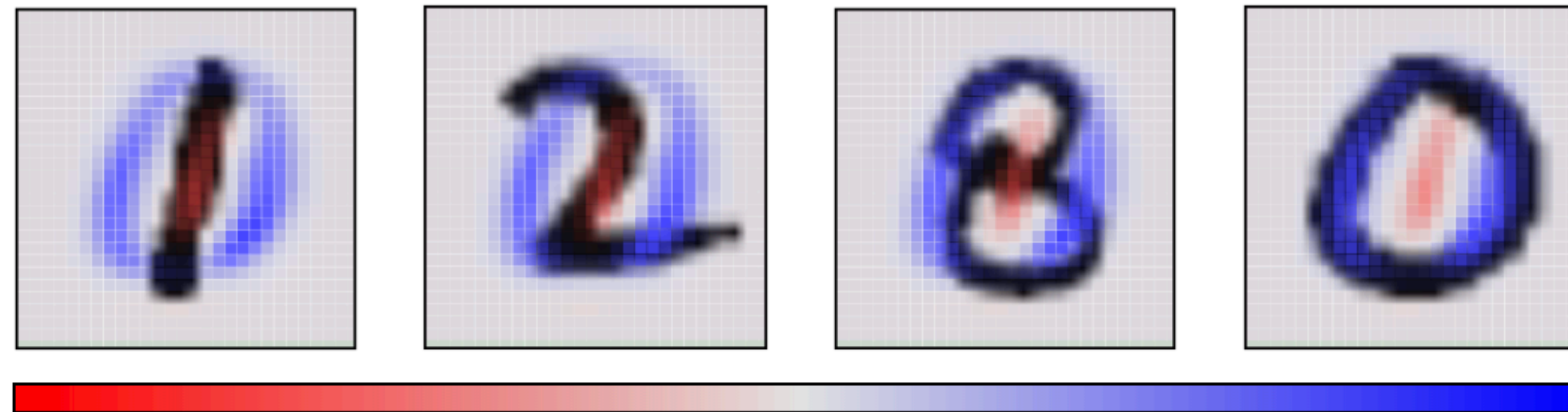
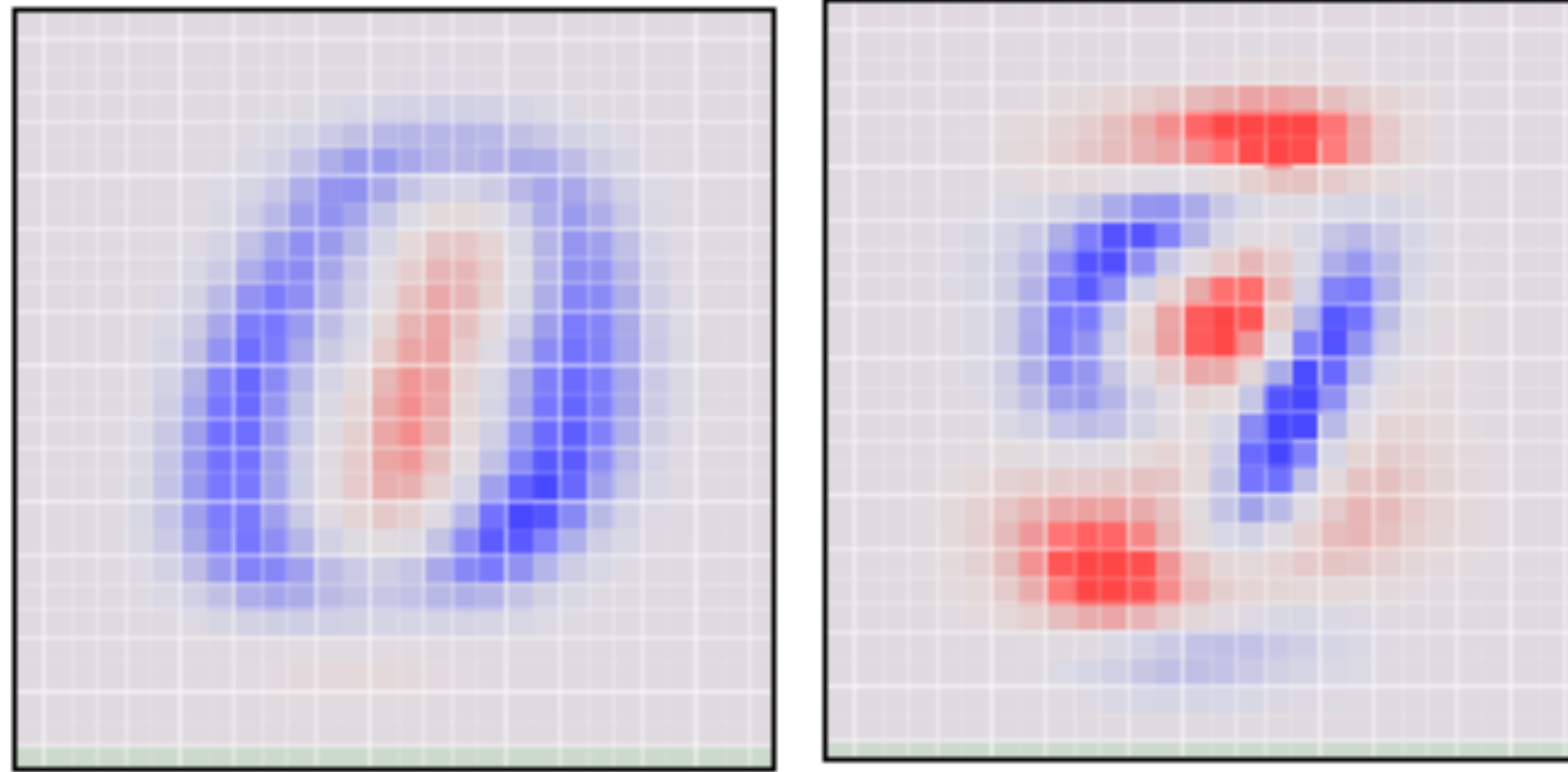


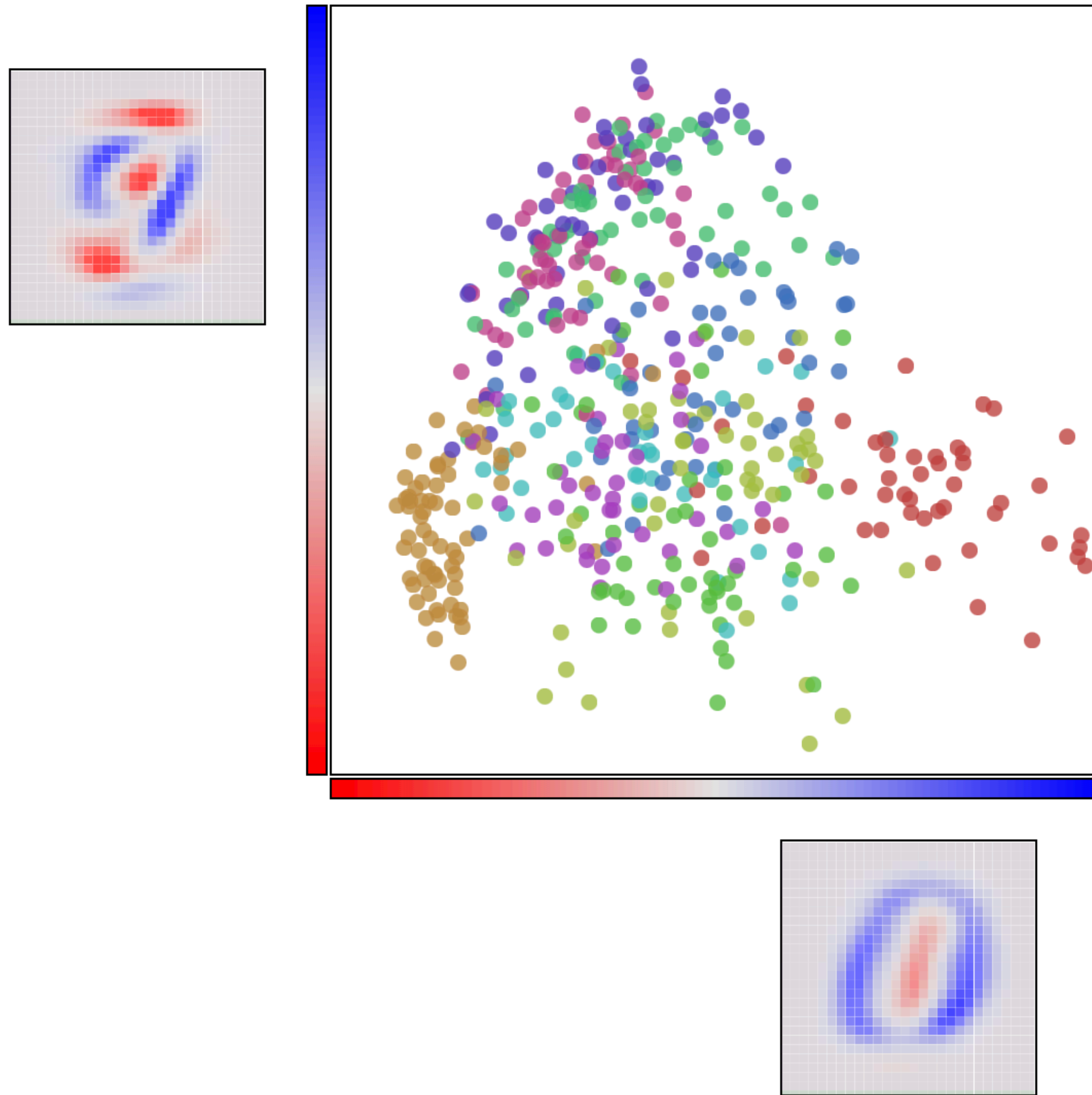
diagram of disentanglement learning



데이터 이해하기 - PCA

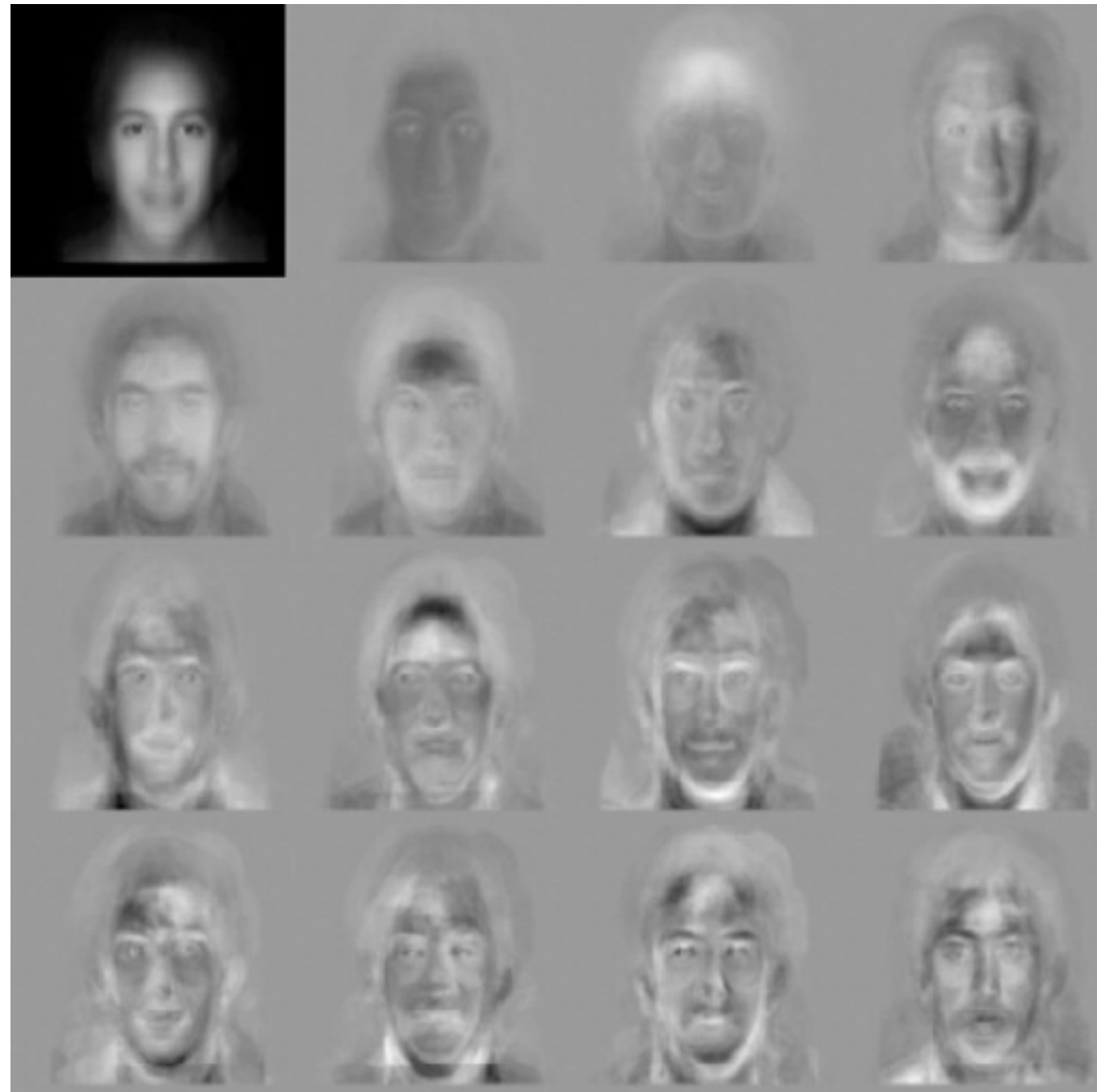


PCA

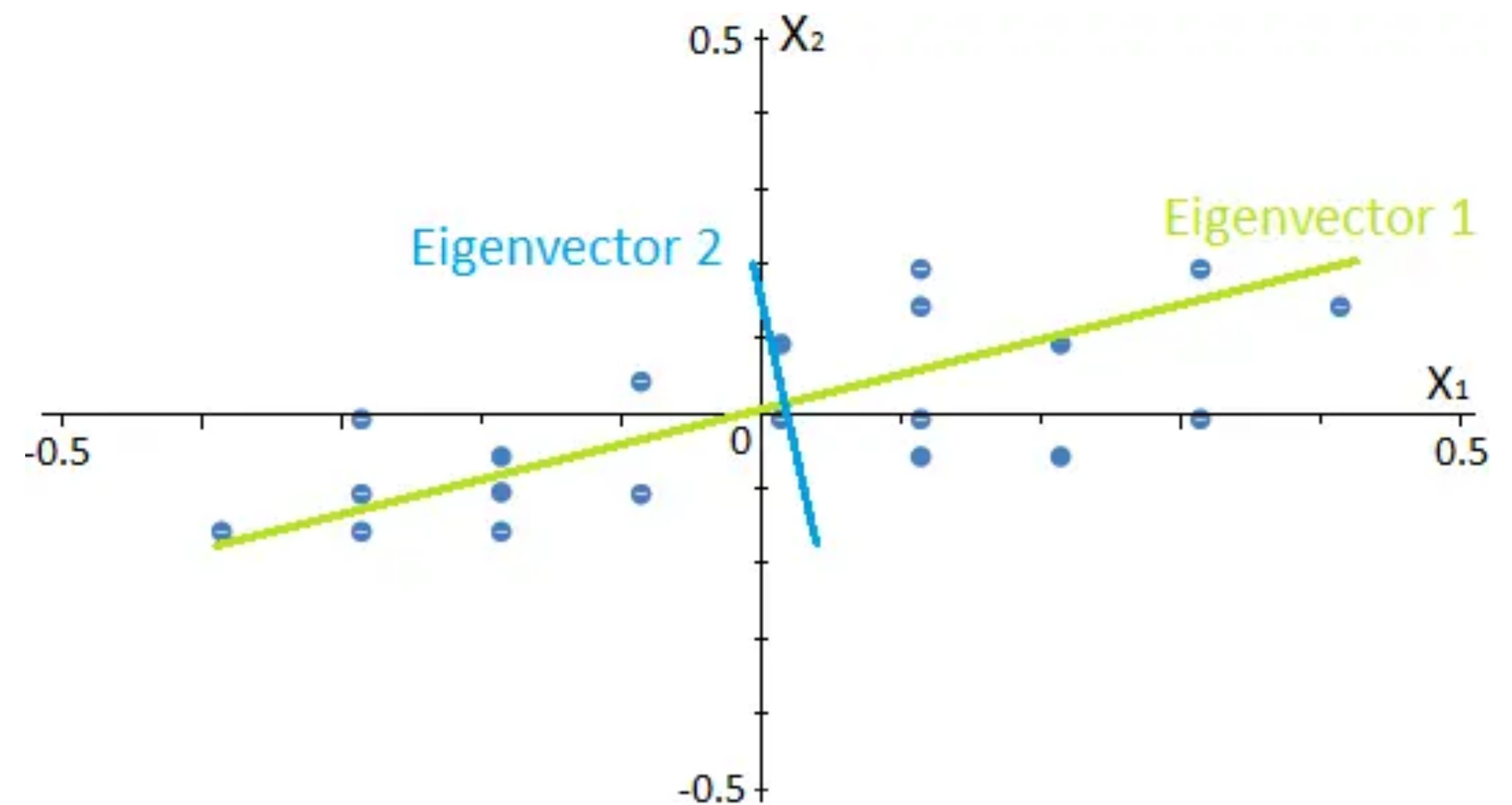


Visualizing MNIST with PCA

feature!!!이요



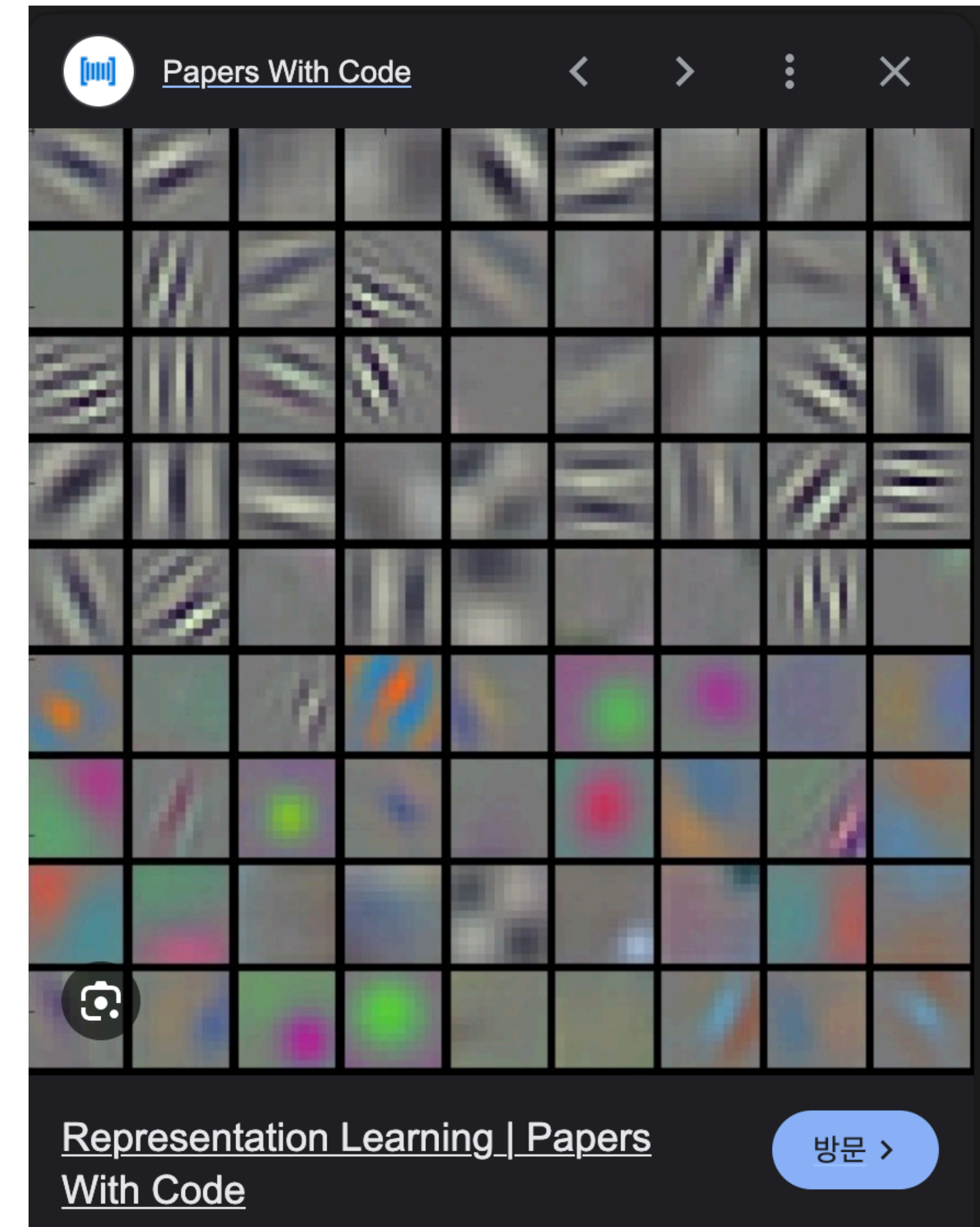
eigenface



capture high variance -> globally!!

So why convolution?

manifold에 대한 믿음. (프랙탈은 아니겠지)



So why convolution in Vision?

The Unbearable Slowness of Being: Why do we live at 10 bits/s?

Jieyu Zheng and Markus Meister
Division of Biology and Biological Engineering
California Institute of Technology
{jzzheng,meister}@caltech.edu

<https://arxiv.org/pdf/2408.10234>

정보 raw input : 1Gbit/s

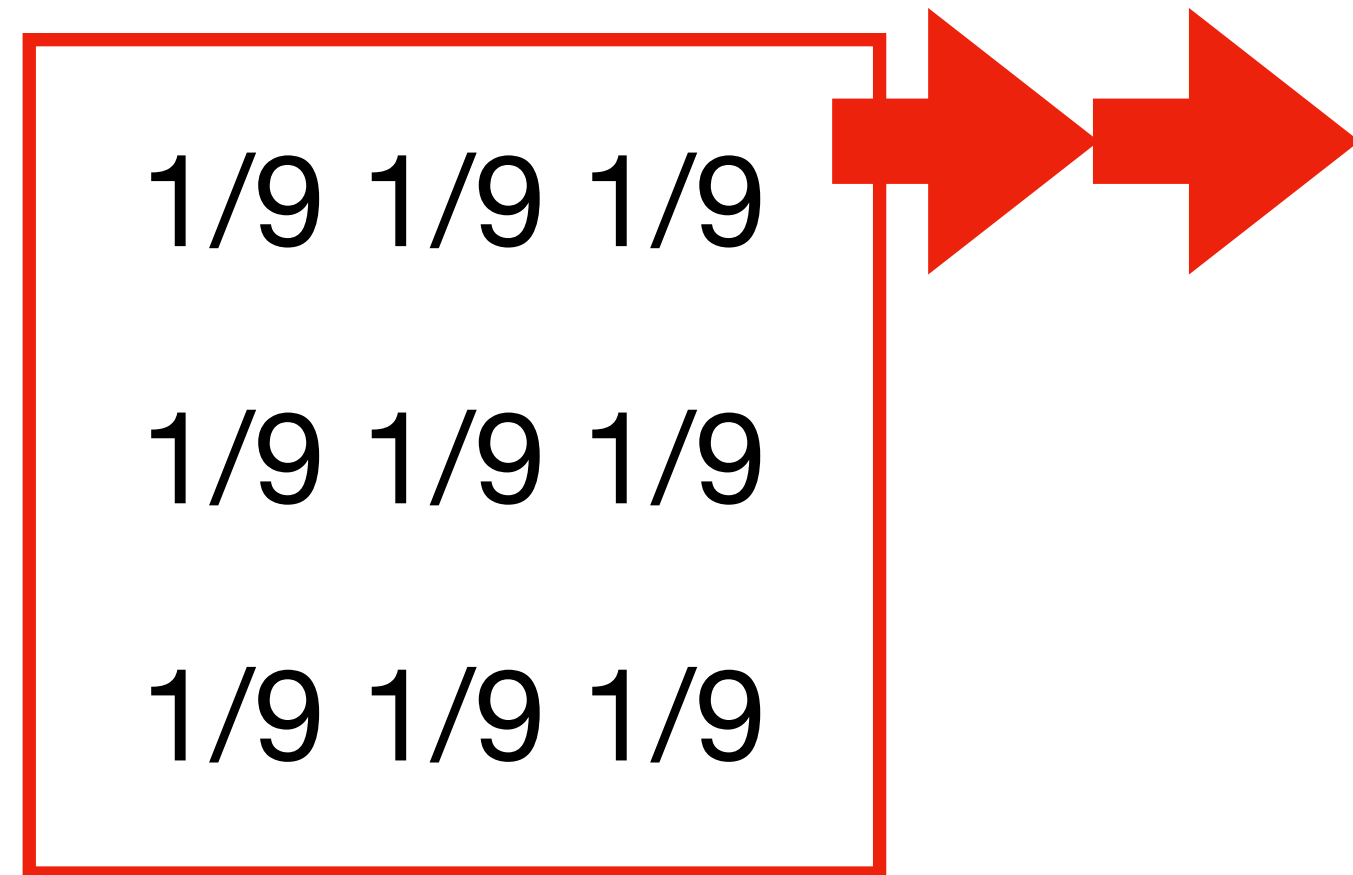
행동 output : 10bit/s (스무고개 등 사례에서)

정보의 양을 줄여나가는 연결!

(무의식의 힘)

= Convolution!

CNN for _____



2칸씩 이동하는 것? = stride

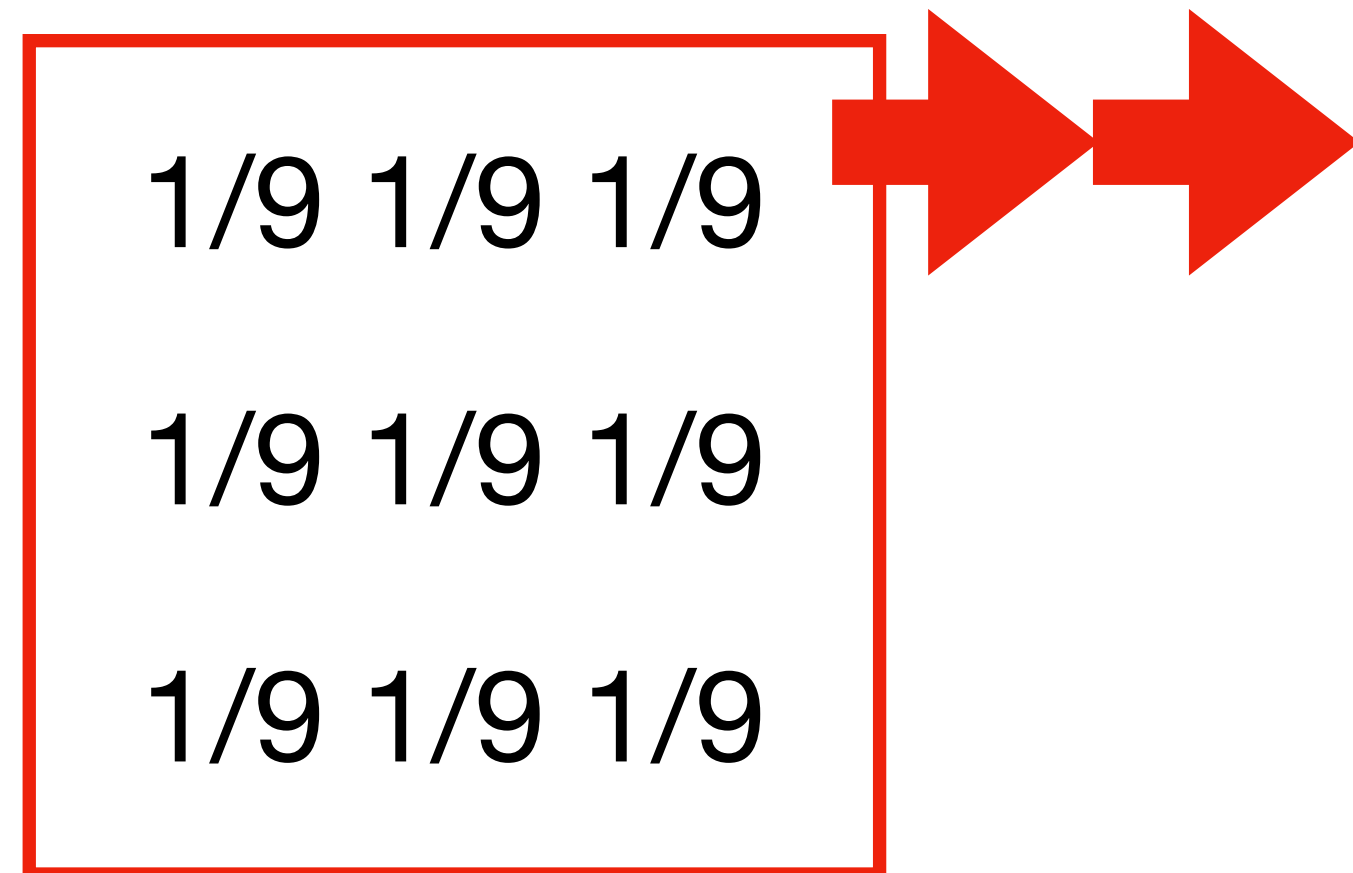
$$\text{sum } f(t)g(n-\text{dilation} * t)$$

$1/9$	(0)	$1/9$	(0)	$1/9$
(0)	(0)	...		
$1/9$		$1/9$		$1/9$
$1/9$		$1/9$		$1/9$

필터 사이에 거리를 두자? = dilation

~~사실 5x5 필터에 0이 채워진 것과 동치이지 않나..~~

CNN for _____



2칸씩 이동하는 것? = stride

kernal size 3x3

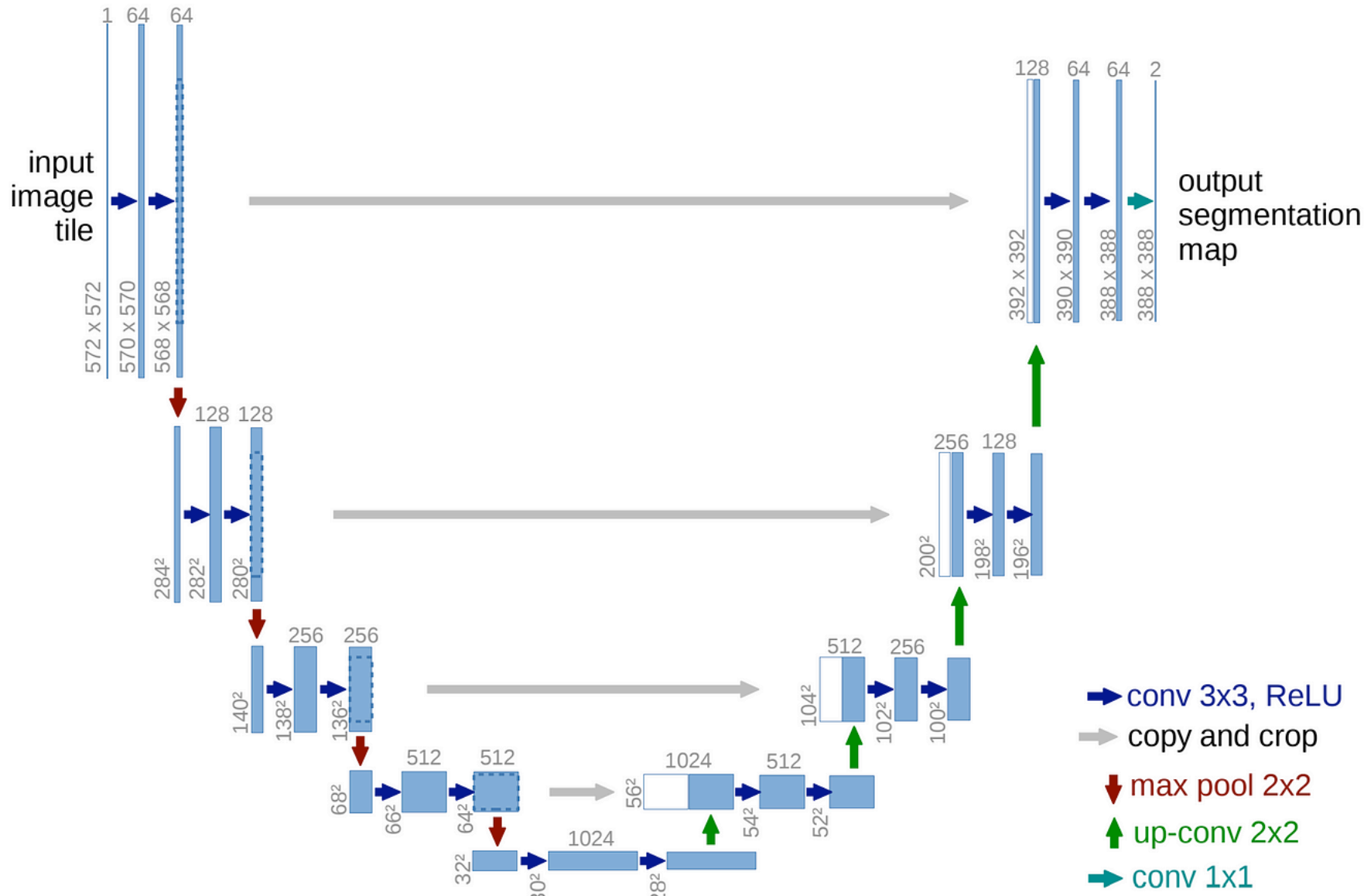
padding 1

stride 2

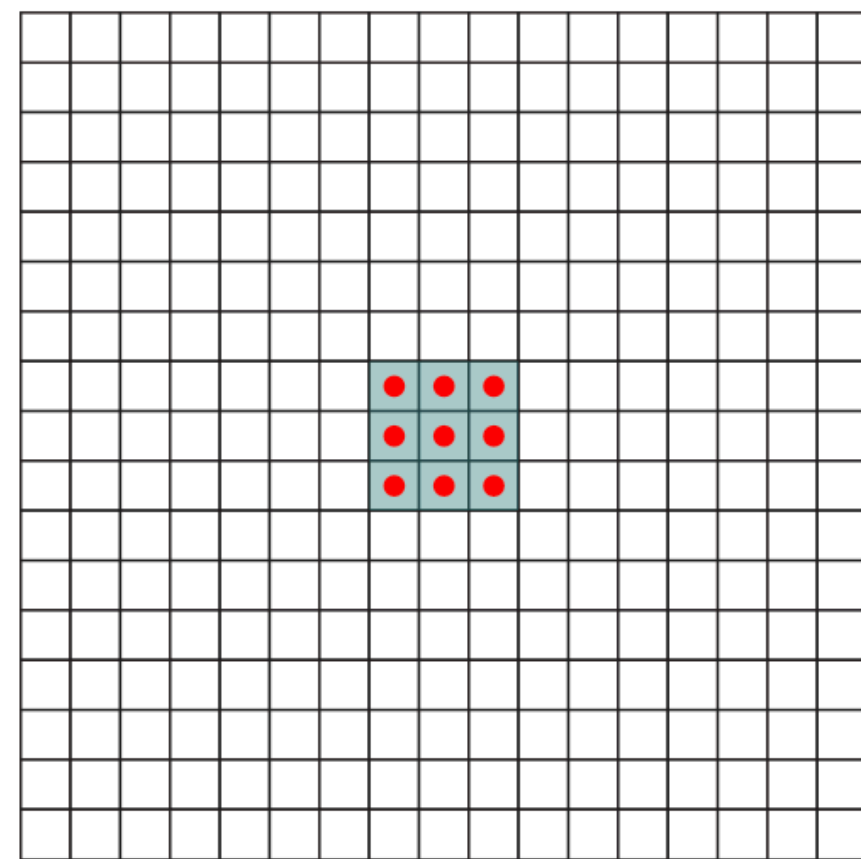
이미지 크기가 1/4 (양변 1/2씩)

(max pool2d 정보 손실)

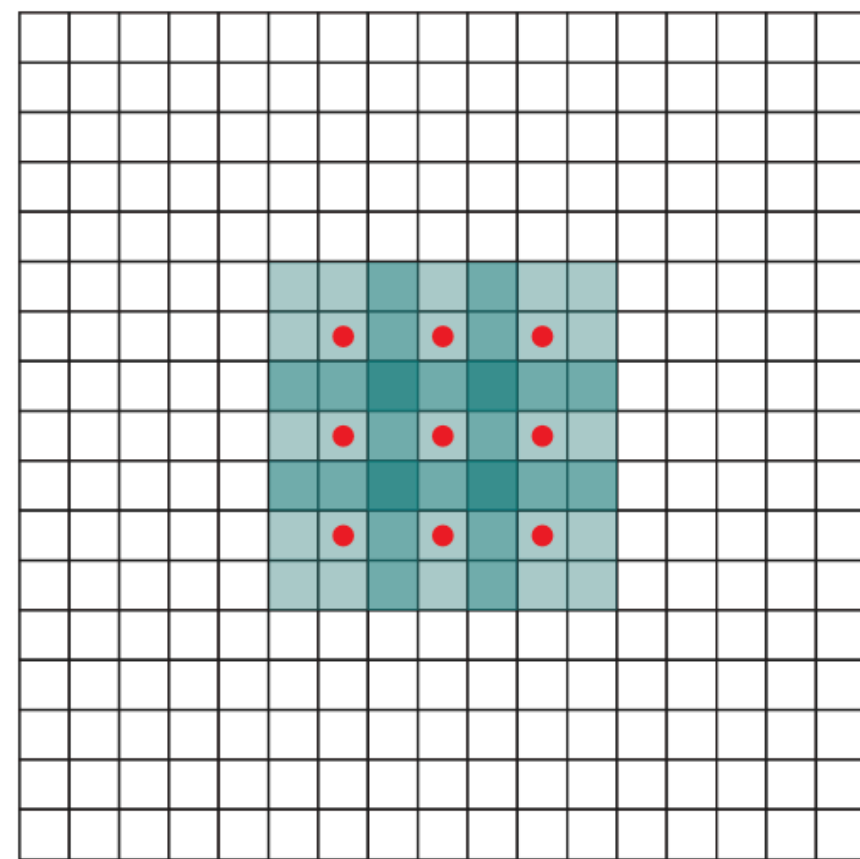
CNN for _____ for Dense Prediction



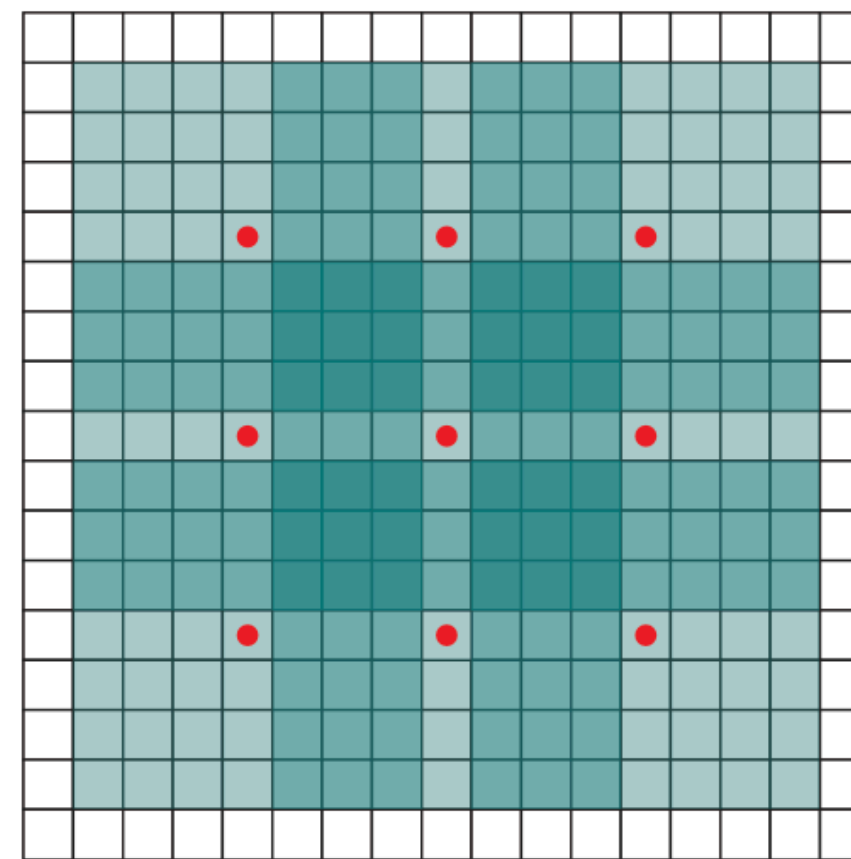
CNN for _____ for Dense Prediction



(a)



(b)



(c)

MULTI-SCALE CONTEXT AGGREGATION BY
DILATED CONVOLUTIONS

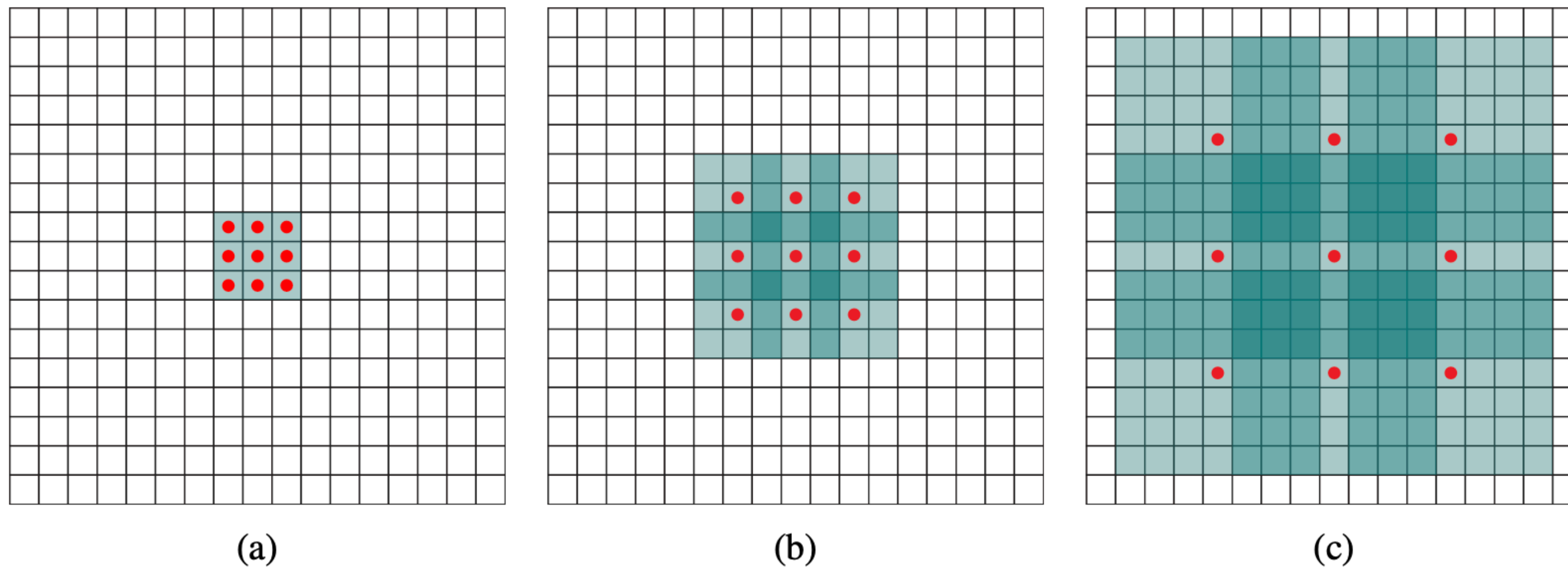
$$\text{sum } f(t)g(n - \text{dilation} * t)$$

필터 사이에 거리를 두자? = dilation

~~사실 5x5 필터에 0이 채워진 것과 동치이지 않나..~~

1/9	(0)	1/9	(0)	1/9
(0)	(0)	...		
1/9		1/9		1/9
1/9		1/9		1/9

CNN for _____ for Dense Prediction



exponentially expanding receptive fields

without losing resolution or coverage

MULTI-SCALE CONTEXT AGGREGATION BY
DILATED CONVOLUTIONS

CNN for _____ for Dense Prediction

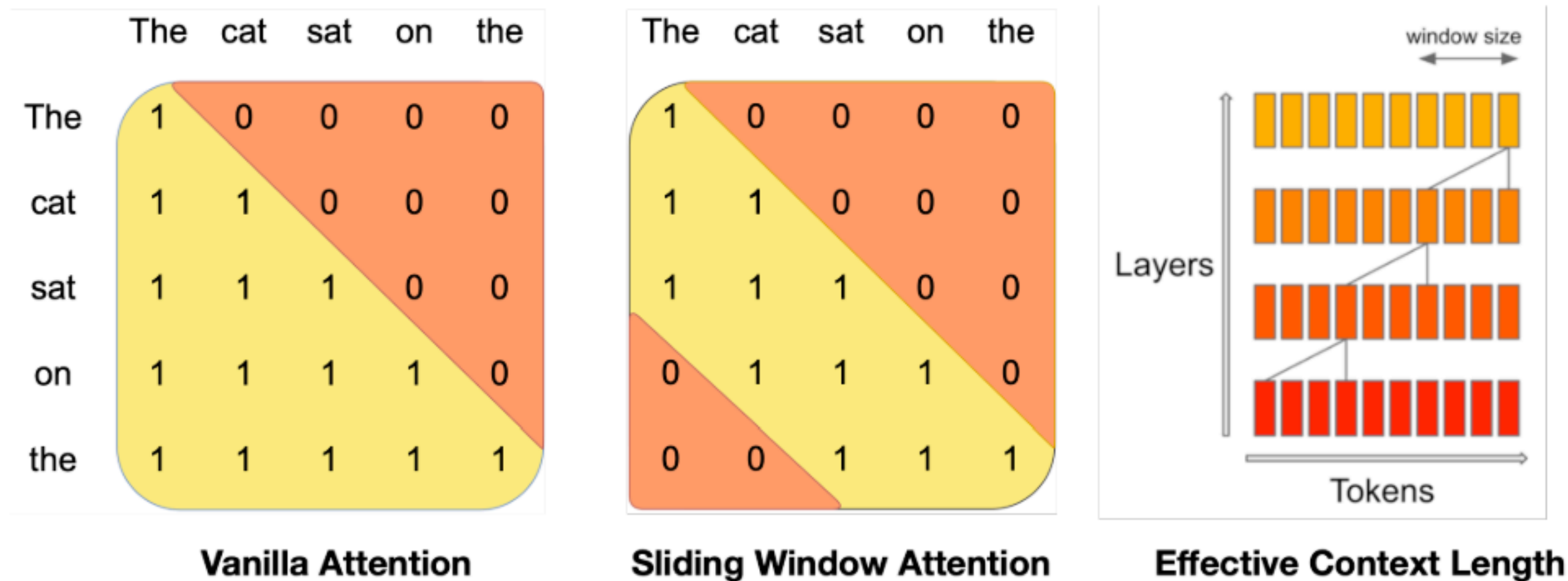


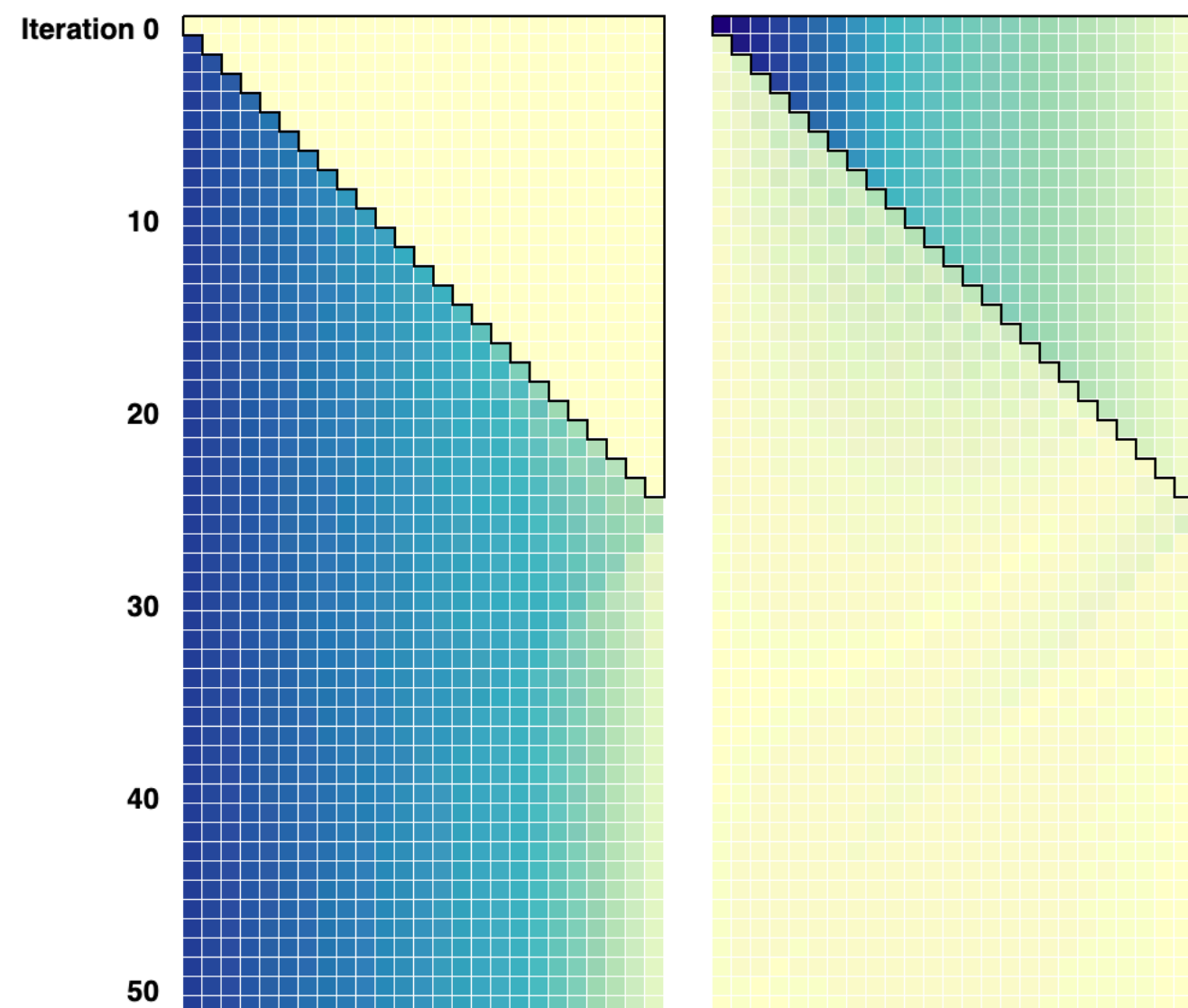
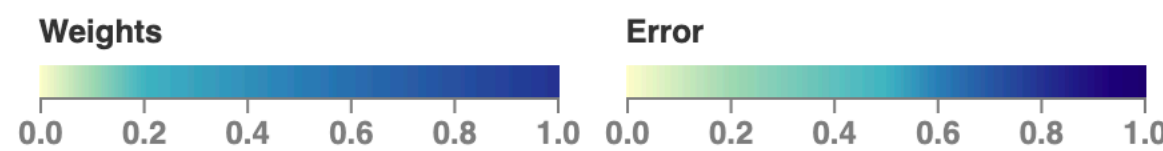
Figure 1: Sliding Window Attention. The number of operations in vanilla attention is quadratic in the sequence length, and the memory increases linearly with the number of tokens. At inference time, this incurs higher latency and smaller throughput due to reduced cache availability. To alleviate this issue, we use sliding window attention: each token can attend to at most W tokens from the previous layer (here, $W = 3$). Note that tokens outside the sliding window still influence next word prediction. At each attention layer, information can move forward by W tokens. Hence, after k attention layers, information can move forward by up to $k \times W$ tokens.

Dilation on Attention Map!

- Attention is \$\$\$\$\$\$
- Propagate the info through layer

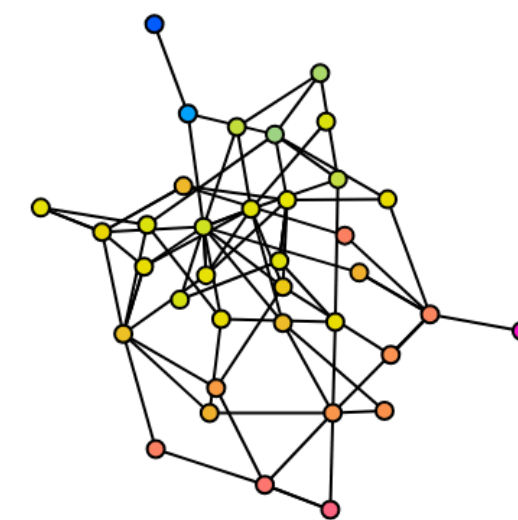
CNN for _____ for Dense Prediction

Here we see the first 50 iterates of momentum on the Convex Rosenbrock for $n = 25$. The behavior here is similar to that of any Linear First Order Algorithm.

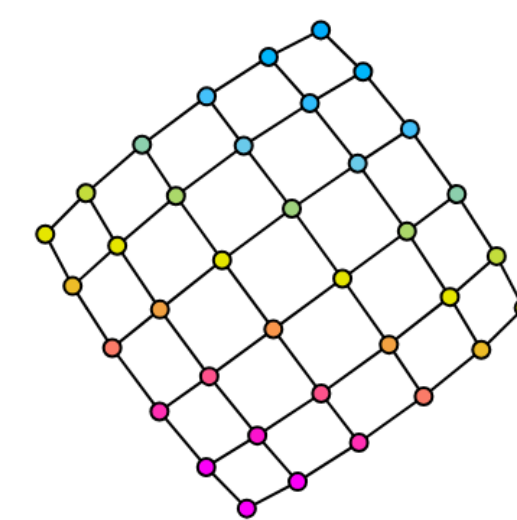


This triangle is a “dead zone” of our iterates. The iterates are always 0, no matter what the parameters.

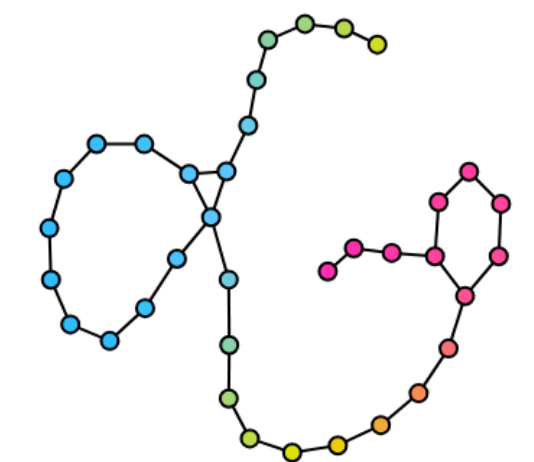
The remaining expanding space is the “light cone” of our iterate’s influence. Momentum does very well here with the optimal parameters.



Small world graphs, like expanders and dense graphs, have excellent conditioning



The conditioning of grids improves with its dimensionality.



And long, wiry graphs, like paths, condition poorly.

정보가 어떻게 전달되는지 그래프의 관점에서 바라본 재밌는 예시

<https://distill.pub/2017/momentum/>

CNN for _____ for Dense Prediction

Sequence의 특정한 타임스탬프 t 에서 feature vector를 각 노드로 하는 그래프를 생각해보자.

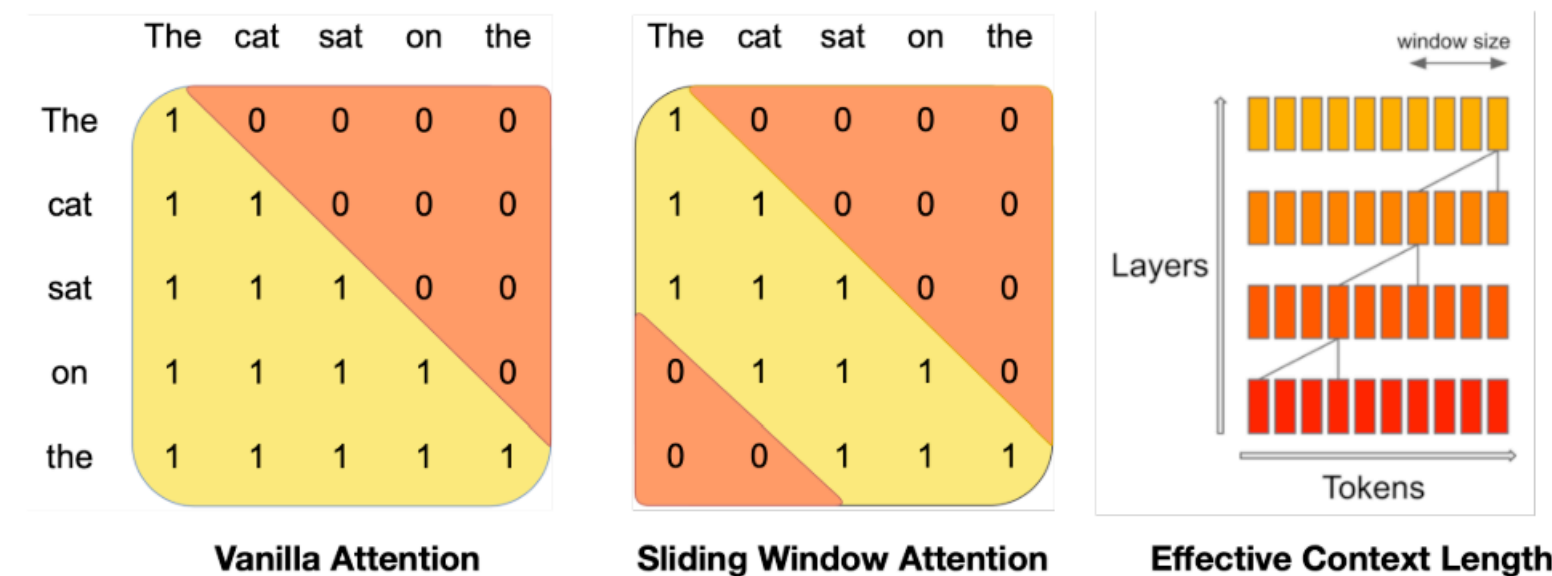


Figure 1: Sliding Window Attention. The number of operations in vanilla attention is quadratic in the sequence length, and the memory increases linearly with the number of tokens. At inference time, this incurs higher latency and smaller throughput due to reduced cache availability. To alleviate this issue, we use sliding window attention: each token can attend to at most W tokens from the previous layer (here, $W = 3$). Note that tokens outside the sliding window still influence next word prediction. At each attention layer, information can move forward by W tokens. Hence, after k attention layers, information can move forward by up to $k \times W$ tokens.

- T 개의 노드
- Vanilla Attention의 경우 $t=0, 1, 2, 3\dots$ 에 따라서 0, 1, 2, 3...개의 연결
- Sliding Window($W=3$) Attention의 경우 $t=0, 1, 2, 3\dots$ 에 따라서 0, 1, 2, 2, 2, 2, 2...

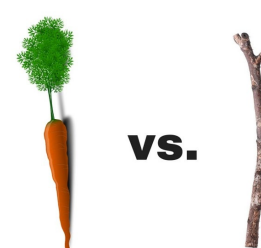
Q: 단어의 종류에 따라서 W 를 다르게 하는게 효율적이지 않나?

이것만으로는 부족하다..!

$$\frac{dLoss}{dx_1} = \frac{dLoss}{dx_{n-1}} \times \dots \times \frac{dx_2}{dx_1}$$



- 모두 1 초과이면 발산!
- 모두 1 미만이면 0

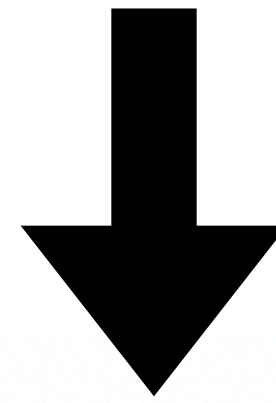


VS.



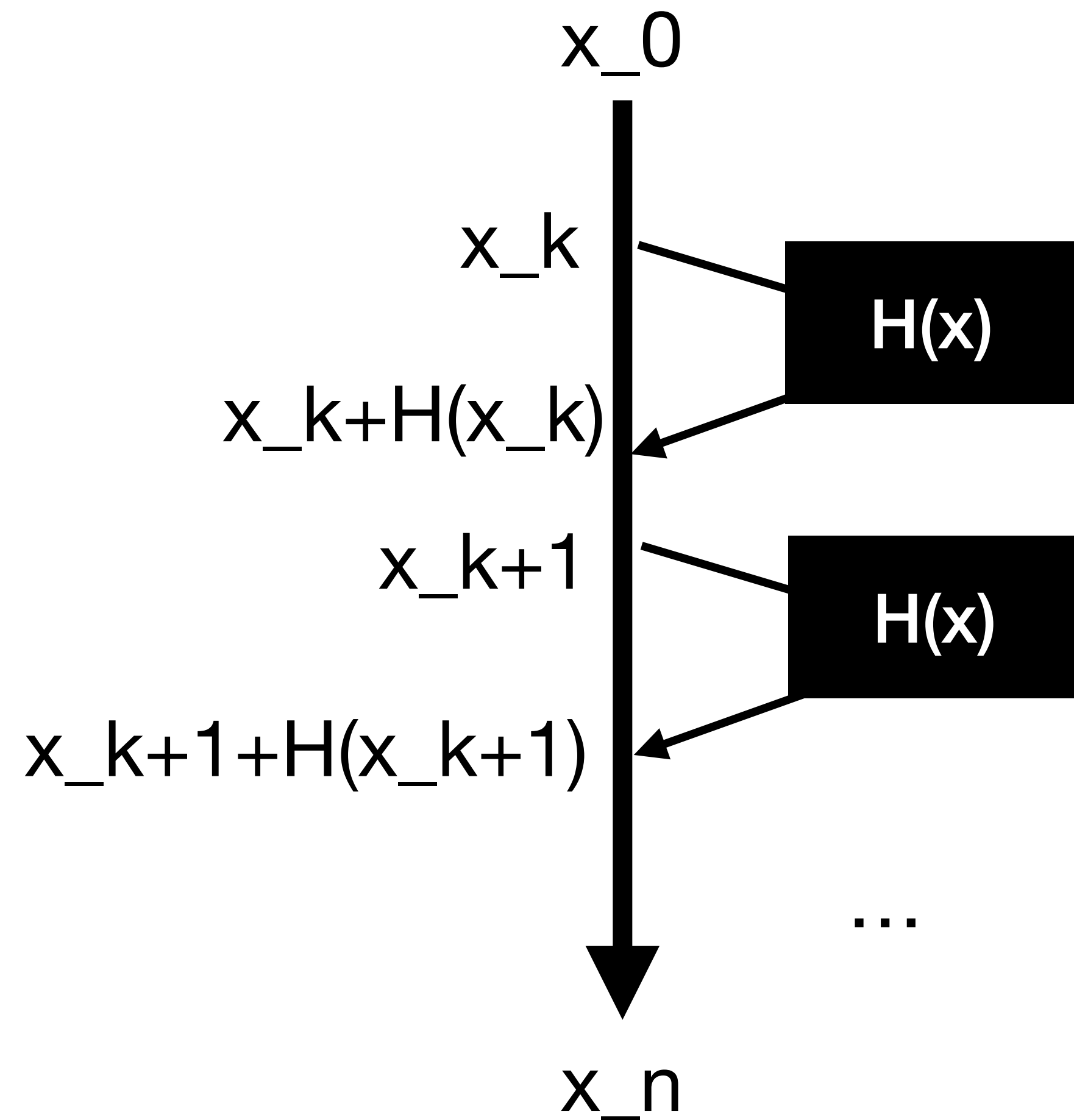
1번 문제에 대하여 - residual mapping

$$x_{n+1} = F(x_n)$$

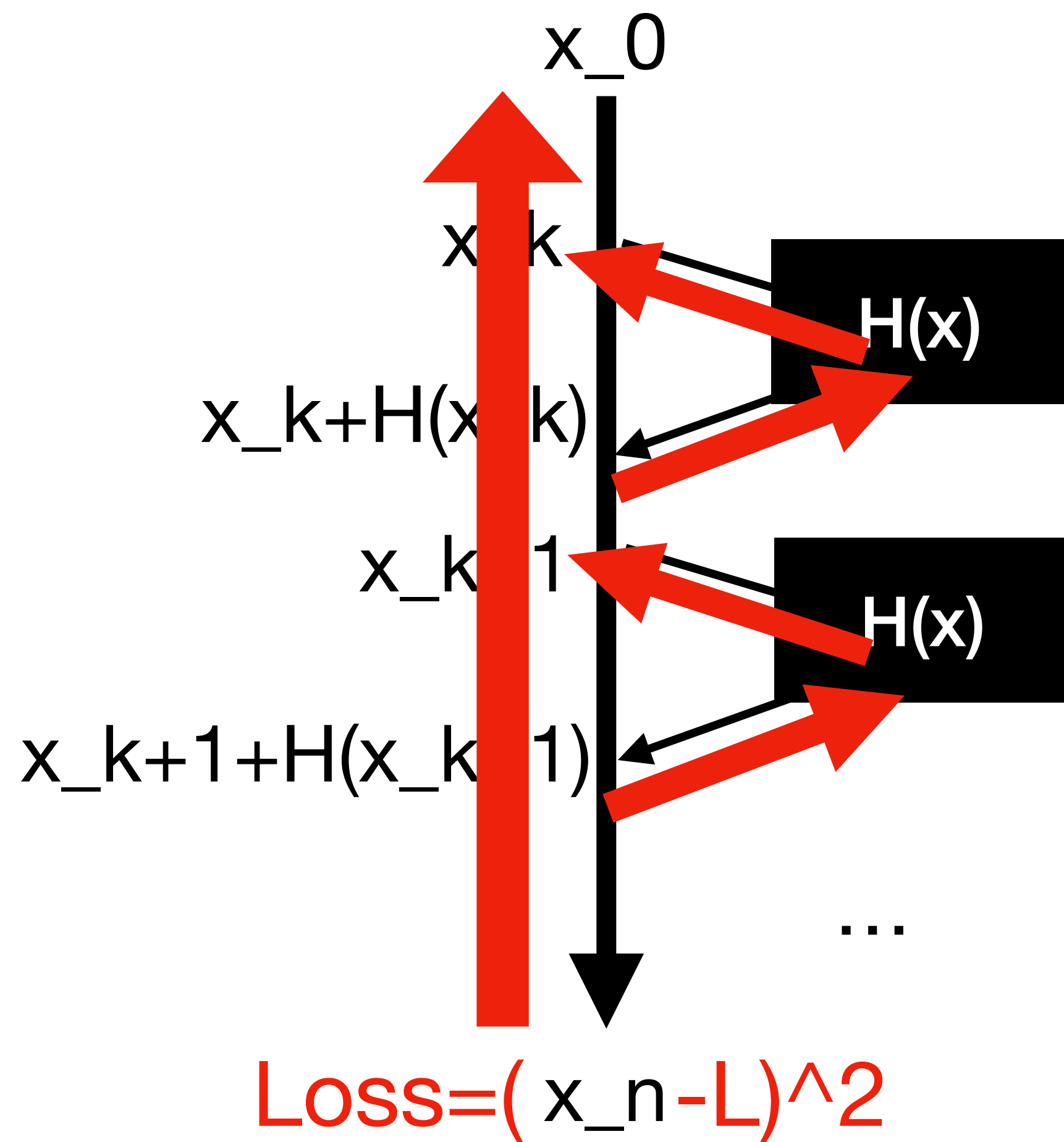


$$x_{n+1} = x_n + H(x_n),$$
$$H(x_n) = x_{n+1} - x_n$$

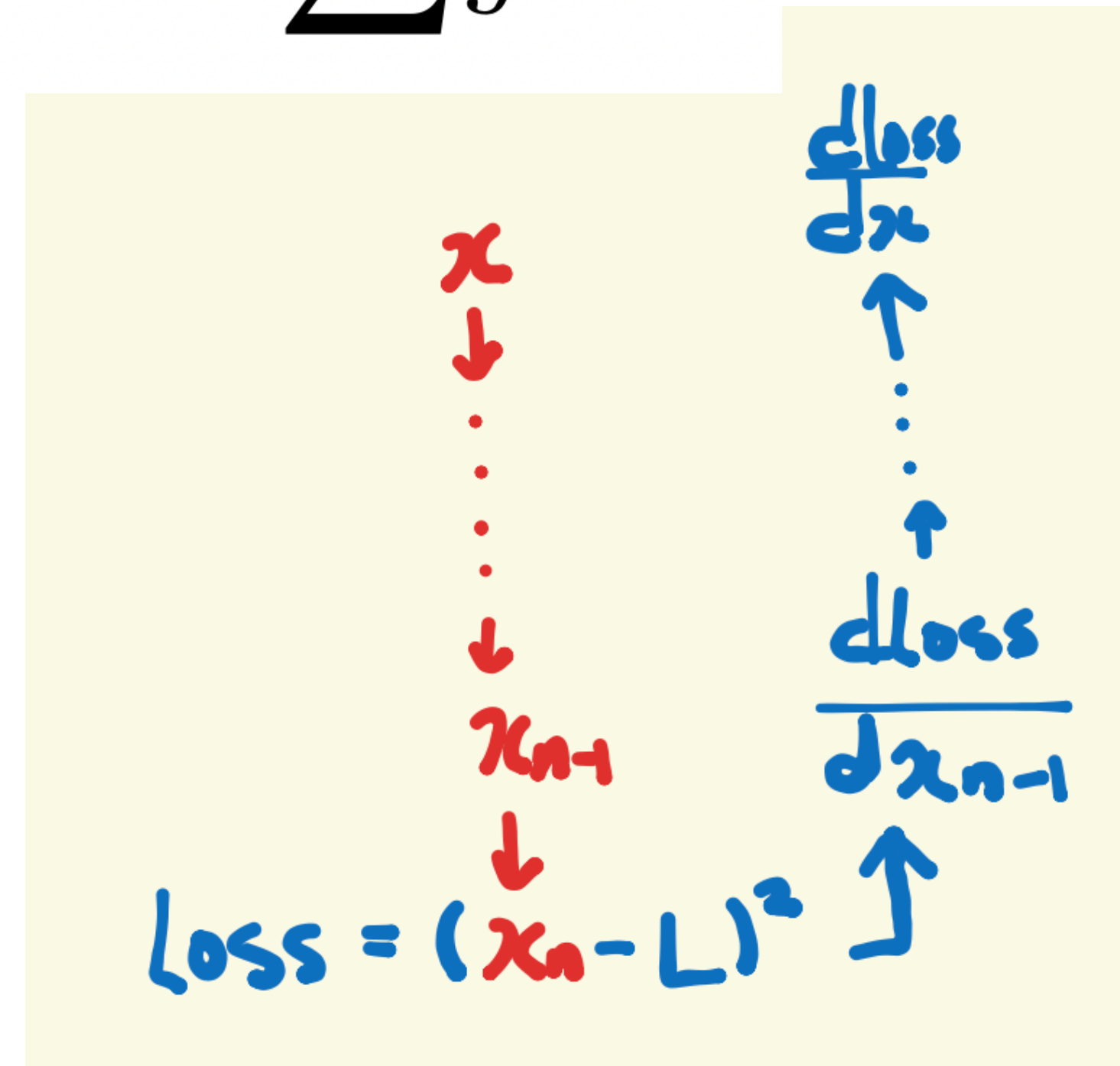
1번 문제에 대하여 - residual mapping



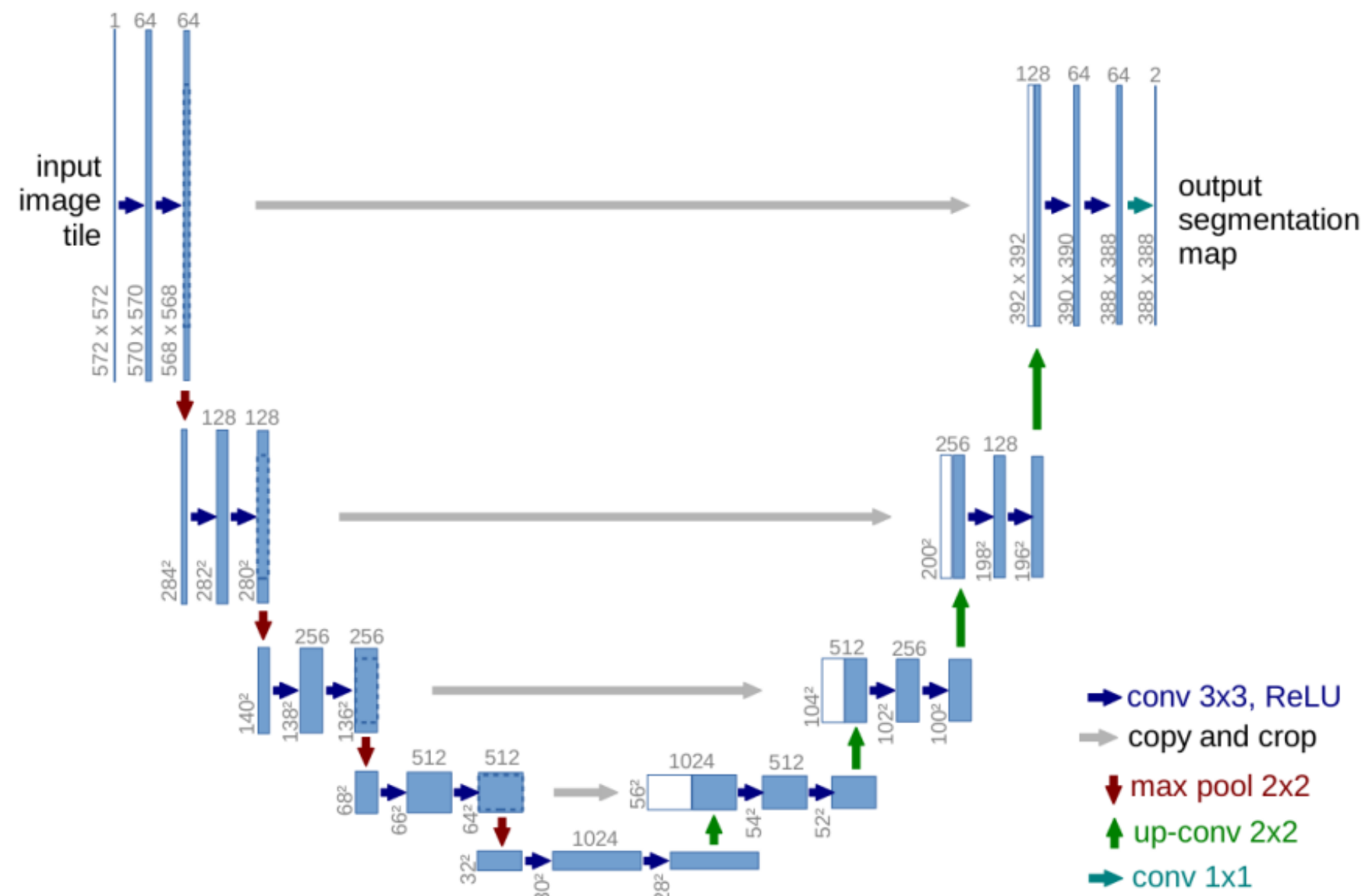
1번 문제에 대하여 - residual mapping



$$\prod grad \rightarrow \sum grad$$



1번 문제에 대하여 - residual mapping



U-Net: Convolutional Networks for Biomedical Image Segmentation

further reading

- Alexnet: <https://karpathy.github.io/2022/03/14/lecun1989/>
- Early deeplearning without CUDA: exploit...
 - <https://www.perplexity.ai/search/please-let-me-know-the-case-in-0yL55SLBSjyEmFddC1kAkw#3>
- 딥러닝을 본격적으로 사용하기 이전에 훈련대신 인간이 작성한 방법론들에 대한 찬양... (CS231n에서 찾아보세요!)